

长文本事实卫士 —— 智能文档一致性检测与协作系统

PatPat-Inconsistency-Hunter

- 组内统一分数
-

目录

- 一、项目概述
 - 1.1 项目背景与痛点
 - 1.2 核心目标
 - 1.3 核心特点
- 二、系统架构设计
 - 2.1 整体架构
 - 2.2 云原生组件说明
 - 2.3 数据流向
- 三、技术栈
 - 3.1 后端技术栈
 - 3.2 前端技术栈
 - 3.3 云原生部署
- 四、核心功能实现
 - 4.1 事实提取
 - 4.2 冲突检测
 - 4.3 溯源校验
 - 4.4 图片内容识别
 - 4.5 协作编辑
- 五、LangGraph 智能体设计
 - 5.1 工作流程架构
 - 5.2 节点定义与状态流转
 - 5.3 容错与回滚机制
- 六、Prompt 工程

- [6.1 事实提取 Prompt](#)
- [6.2 冲突检测 Prompt](#)
- [6.3 溯源验证 Prompt](#)
- [6.4 幻觉防护策略](#)
- [七、数据库设计](#)
 - [7.1 PostgreSQL 表结构](#)
 - [7.2 Redis 数据结构](#)
- [八、Docker 容器化部署](#)
 - [8.1 服务编排](#)
 - [8.2 部署流程](#)
- [九、前端界面展示](#)
 - [9.1 首页](#)
 - [9.2 文档分析页](#)
 - [9.3 分析结果页](#)
 - [9.4 协作空间](#)
 - [9.5 统计仪表盘](#)
- [十、API 接口设计](#)
- [十一、项目分工](#)
- [十二、总结与展望](#)

一、项目概述

1.1 项目背景与痛点

1.1.1 长文档多人协作带来的事实失序问题

在学术论文、可行性报告、技术方案等长文档的撰写过程中，多人协作和分章节生成已成为常见模式。文档往往由不同作者在不同时间、不同上下文假设下分别完成，再被合并为一个整体。随着篇幅扩大和修改轮次增多，文档内部逐渐形成“局部自治、全局失序”的状态，事实一致性问题随之频繁出现。这些问题通常并非单点错误，而是跨章节、跨位置的隐性冲突，例如：

- 同一核心数据在前后章节出现不同数值，难以判断最终口径；
- 关键时间节点在摘要与正文中描述不一致，破坏整体逻辑；
- 人员、机构或角色信息在不同位置发生错乱，影响专业可信度；
- 图表数据与文字结论相互矛盾，使读者对结果产生怀疑。

在多人协作场景下，每位作者通常只对自己负责的章节内容高度熟悉，却难以持续感知全文范围内已经出现过的事实描述。当某个事实在后续章节被“合理修改”时，前文却未同步更新，冲突便悄然产生。这类问题往往在写作阶段难以及时暴露，却会在评审、答辩或对外发布时集中显现，带来较高的返工成本。

1.1.2 传统人工审核方式成本高、覆盖有限

面对五千字以上、甚至数万字规模的长文档，传统的人工审核方式高度依赖审阅者的经验、记忆力和耐心。审核人员需要在通读全文的过程中，持续对关键事实进行跨章节比对，本身就是一项高认知负担、低效率的工作。即便借助关键词搜索、批注或简单的重复检测工具，也通常只能发现显性的重复或明显错误，而对于以下情况则往往力不从心：

- 同一事实以不同表述方式出现，难以通过简单搜索发现；
- 隐含在结论、推导或图文关系中的逻辑矛盾；
- 跨章节、跨版本逐步演化形成的事实偏移。

随着大语言模型被广泛引入写作流程，文档生成效率显著提升，但事实一致性并未同步得到保障。模型在不同上下文中生成的内容缺乏全局约束，使得人工审核的压力进一步放大。在现实项目中，全面、系统地依靠人工手段保证长文档事实一致性，已经变得不具备可扩展性。

因此，如何通过自动化手段在文档层面持续抽取、登记和比对事实，并在发现冲突时提供明确的定位与修正依据，成为长文档协作写作中亟需解决的核心问题。本项目正是针对这一痛点，尝试引入“事实卫士”这一中间校验智能体，为长文本提供可持续、工程化的一致性保障。

1.2 核心目标

本项目的核心目标是构建一个面向长文本的一致性校验智能体，作为内容生成、协作编辑与最终交付之间的“中间件”，在不干扰原有写作流程的前提下，为文档提供持续、自动化的事实一致性保障。系统重点围绕长文档、高并发协作与复杂事实关系，解决传统人工审核难以规模化的问题，具体目标包括：

- **自动化事实提取**：面向五千字以上的长文档，自动从全文中抽取结构化事实信息，包括数值、时间、人物、实体关系、结论性陈述等，并将其统一登记到事实黑板中。通过结构化表示替代人工记忆，为后续一致性分析提供可靠基础。
- **全文范围的冲突检测与定位**：在文档全局范围内对已抽取的事实进行对比分析，识别跨章节、跨位置的前后不一致描述。系统不仅判断是否存在冲突，还会对冲突类型进行分类（如数值冲突、时间冲突、实体冲突、逻辑冲突、图文冲突等），并精确定位冲突发生的位置，降低人工排查成本。
- **冲突事实的溯源校验与修正建议**：对检测到的冲突事实，系统支持进一步的溯源验证流程，通过外部信息检索或上下文证据分析，对事实的真实性进行校验。在此基础上，结合冲突上下文生成可解释的修正建议，辅助用户判断应保留、修订或统一的事实版本，而非仅给出“是否冲突”的结果。
- **面向协作场景的实时一致性保障**：支持多人同时编辑长文档，通过实时协同机制与章节级锁定策略，确保协作过程的可控性。在协作写作过程中，系统能够在保存或触发检测时进行实时冲突分析，并将结果推送给相关协作者，使事实一致性问题尽可能在写作阶段被发现和修正。

- **可视化分析与结果呈现**：通过仪表盘和分析视图，对文档中的事实规模、冲突数量、冲突类型分布及严重程度进行可视化展示，帮助用户从整体层面理解文档质量。同时支持分析结果的多格式导出，满足审核、评估与留档等实际使用需求。

通过上述目标的实现，本项目旨在将“事实一致性校验”从依赖人工经验的事后审查，转变为贯穿长文档写作与协作全过程的基础能力，为高质量长文本的生成与交付提供稳定支撑。

1.3 核心特点

PatPat-Inconsistency-Hunter是一个基于 LLM Agent 的长文档事实一致性检测与协作平台，围绕“长文本、多事实、多协作者”的复杂写作场景进行设计。系统以“事实一致性中间件”为核心定位，将文档解析、事实抽取、冲突推理、协作编辑与结果可视化整合为一条可观测、可扩展的智能分析链路。

1.3.1 多格式、多模态的统一输入处理

系统支持 TXT、Markdown、DOCX、PDF 等多种常见长文档格式，并在分析前统一解析为内部的富文本结构表示。对于包含图表和图片的文档，系统会自动抽取图片内容，并通过视觉模型生成可分析的文本描述，从而避免图文信息割裂带来的事实盲区，实现真正的“全文一致性”检测。

1.3.2 基于 LLM Agent 的智能分析流水线

核心分析能力由大语言模型驱动，并通过 LangGraph 对整个分析过程进行图谱化编排。文档解析、事实抽取、事实过滤、冲突推理、溯源验证等步骤被拆分为可观测、可回退的节点，使复杂分析过程具备工程可控性，而非一次性“黑盒推理”。

1.3.3 面向长文档的事实黑板与冲突推理机制

系统引入“事实黑板”作为中间态存储，将从全文中抽取的关键事实进行统一注册和管理。在此基础上，对事实进行全局比对，而非局限于相邻段落或局部文本，从而能够识别跨章节、跨位置的隐性冲突。冲突结果不仅给出是否不一致，还会结合冲突类型和上下文进行严重度分级，便于用户判断处理优先级。

1.3.4 协作友好的实时一致性保障

针对多人协作写作场景，系统内置实时协同编辑与章节级锁定机制，通过 WebSocket 实现内容与检测结果的即时同步。在多人同时编辑的情况下，系统能够协调编辑权限、避免相互覆盖，并在保存或触发检测时，将冲突结果定向推送给相关协作者或房主，使事实问题尽早暴露，而非事后集中爆发。

1.3.5 云原生架构与可视化结果呈现

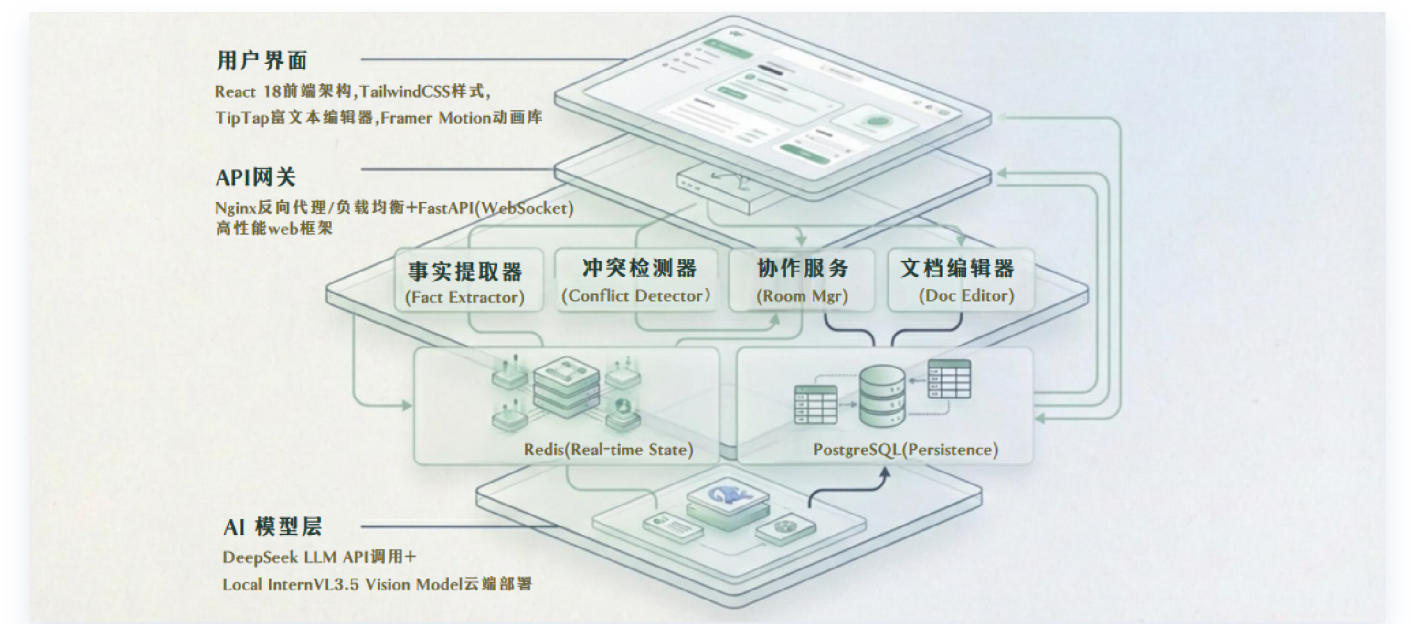
系统采用云原生架构设计，基于 Docker 进行整体部署，Redis 作为统一的状态中心与事实/冲突黑板，PostgreSQL 负责分析结果与历史数据的持久化。在此基础上，通过可视化仪表盘对分析结果进行聚合展示，包括冲突数量、类型分布、严重度趋势等指标，帮助用户从整体层面评估文档质量，并支持多格式报告导出以满足审核与交付需求。

通过上述核心特点，PatPat-Inconsistency-Hunter 不仅是一个“发现错误”的工具，更是一个将事实一致性能力系统化、流程化、可协作化的智能基础设施。

二、系统架构设计

2.1 整体架构

系统整体采用分层式架构设计，自上而下依次为**用户界面层**、**API 接入层**、**核心业务服务层**、**状态与存储层**以及**AI 模型层**。各层之间通过清晰的数据流与职责边界进行解耦，使系统既能够支撑复杂的长文档智能分析，又能在多人协作与高并发场景下保持稳定运行。



从整体上看，架构以“文档→事实→冲突→协作→状态”为主线，围绕事实一致性这一核心能力展开。

2.1.1 用户界面层

最上层为用户界面层，负责承载所有交互与可视化能力。前端基于 React 18 构建，结合 TailwindCSS 提供统一的视觉风格，并通过 TipTap 富文本编辑器支持结构化的长文档编辑。Framer Motion 用于增强界面状态切换与交互反馈，使复杂分析与协作过程对用户而言保持可感知、可理解。

用户在该层完成文档编辑、协作写作、冲突查看以及分析结果浏览，所有操作均通过标准 HTTP 或 WebSocket 与后端进行通信。

2.1.2 API 网关与通信层

用户界面之下是统一的 API 接入层。系统使用 Nginx 作为反向代理与流量入口，对外统一暴露服务地址，并将请求转发至 FastAPI 后端。FastAPI 不仅承担传统 REST API 的请求处理，还通过 WebSocket 提供实时通信能力，用于：

- 多人协作编辑时的内容同步
- 任务分析进度的实时推送
- 冲突检测结果的即时反馈

这一层将前端与后端业务逻辑彻底解耦，同时为高并发和长连接场景提供稳定支撑。

2.1.3 核心业务服务层

架构的中间层是系统的核心业务能力区，围绕长文档一致性分析与协作场景拆分为多个职责明确的服务模块：

- **事实提取器**：负责从解析后的文档内容中抽取结构化事实，包括数值、时间、实体、结论性描述等，为后续一致性分析提供基础数据。
- **冲突检测器**：基于已登记的事实进行全局对比，识别跨章节、跨位置的事实不一致，并对冲突类型与严重程度进行判定。
- **协作服务**：管理多人协作写作场景，包括房间、成员、章节权限、实时状态同步等，确保协作过程可控且可追溯。
- **文档编辑器服务**：负责文档内容的存储、版本管理以及协作场景下的编辑同步，与前端富文本编辑器形成闭环。

这些服务并非孤立运行，而是通过共享状态与统一的数据通道进行协作，构成完整的业务处理链路。

2.1.4 状态与存储层

在核心业务服务之下，系统引入双存储体系以应对不同类型的数据需求。

Redis 作为实时状态中心，承载高频、易变的数据，包括任务状态与进度、事实黑板、冲突黑板、章节锁、检测锁以及 WebSocket 会话信息。通过 Redis，系统能够在并发场景下保证状态一致性，并支持快速读写与实时推送。

PostgreSQL 则用于长期持久化存储，负责保存用户数据、文档内容、分析结果、协作历史与统计信息，确保数据的可靠性与可追溯性。

2.1.5 AI 模型层

最底层是系统的智能能力来源。**DeepSeek LLM API** 提供核心的自然语言理解与推理能力，用于事实提取、冲突判断与文本级分析；**InternVL3.5 Vision Model** 以本地部署方式运行，负责识别文档中的图片和图表内容，将视觉信息转化为可参与一致性分析的文本事实。

通过将模型能力下沉为独立的一层，系统避免了模型逻辑与业务逻辑的强耦合，使分析流程能够通过 workflow 方式灵活编排和扩展。

2.2 云原生组件说明

为支撑长文档一致性分析这一计算密集、状态复杂、协作频繁的应用场景，PatPat-Inconsistency-Hunter 在架构设计上采用了云原生优先的技术选型。系统从一开始就围绕“可部署、可扩展、可观测”的目标进行构建，而不是将其视为单一脚本或离线工具。

在多人协作、实时分析和长任务并存的情况下，系统需要同时满足以下要求：一方面要支持高并发状态同步与实时通信，另一方面又要保证分析结果与历史数据的可靠持久化；同时，还需要具备清晰的服务边界，便于本地部署、调试与后续扩展。基于这些现实需求，本项目在基础设施层面引入了以容器化和服务解耦为核心的云原生组件组合。

下表总结了本系统中各个关键云原生组件的技术选型、核心职责以及它们在整体架构中所体现的云原生特性：

组件	技术选型	职责	云原生特性
容器编排	Docker Compose	服务编排与生命周期管理	容器化部署、服务隔离
反向代理	Nginx	请求路由与流量入口	WebSocket支持、静态资源缓存
状态缓存	Redis 7	事实黑板、冲突黑板、分布式锁	高并发状态同步、低延迟读写
持久化	PostgreSQL 15	文档、分析结果与协作数据存储	事务支持、数据一致性保障
后端框架	FastAPI	异步 API 与实时服务	高性能、自动接口文档生成
前端框架	React 18	用户界面与交互层	组件化、响应式渲染

通过上述组件组合，系统在工程层面实现了“实时状态与长期数据分离”“分析计算与协作控制解耦”的设计目标。Redis 承载高频、易变的中间态数据，PostgreSQL 负责稳定可靠的数据沉淀；Nginx 与 FastAPI 共同构成对外服务入口，为 HTTP 与 WebSocket 提供统一接入；Docker Compose 则将整个系统封装为可一键启动的服务集合，使部署和环境复现成本显著降低。

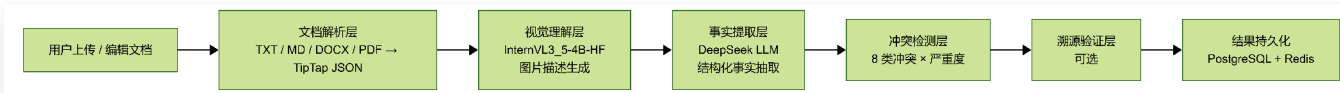
这种云原生架构设计，使 PatPat-Inconsistency-Hunter 能够在本地、服务器或云环境中保持一致的运行行为，也为后续扩展更多分析节点、协作模式和存储后端提供了良好的基础。

2.3 数据流向

在 PatPat-Inconsistency-Hunter 中，长文档的一致性分析并不是一次性的“黑盒处理”，而是一条由多个阶段组成、可观测、可中断的数据处理流水线。文档内容在系统内部会经历解析、理解、抽取、推理与存储等多个步骤，每个阶段都产出明确的中间结果，并通过状态中心进行统一管理。

整体数据流以“用户上传或编辑文档”为起点，最终以“结构化分析结果与可视化呈现”为终点，形成一条清晰、可追踪的分析链路。

2.3.1 数据流处理流程



2.3.2 数据流说明

- **文档解析层**：数据流的第一步从文档解析开始。系统支持多种常见长文档格式，并在进入分析流程前统一解析为内部的 TipTap JSON 结构。该阶段不仅提取纯文本内容，还会同时解析文档中的图片资源，为后续多模态理解打下基础。通过结构化表示，文档内容不再是不可分割的字符串，而是可被精确定位和回溯的分析对象。
- **视觉理解层**：对于包含图表或图片的文档，系统会将图片送入视觉模型进行处理。InternVL3_5-4B-HF 模型会为图片生成对应的语义描述，并将这些描述回填到文档内容中，替换原有的图片占位符。这样，原本仅存在于视觉层面的信息被转化为可参与文本一致性分析的事实来源，避免图文割裂带来的分析盲区。
- **事实提取层**：在获得完整、可分析的文本后，系统调用大语言模型对文档进行事实抽取。模型会识别并结构化输出数值、时间、实体、定义和结论性陈述等关键信息，并将这些事实统一写入事实黑板。事实黑板作为全局中间态存储，使后续分析能够在全文范围内进行，而不是局限于局部上下文。
- **冲突检测层**：基于已登记的事实集合，系统会生成候选事实对并进行两两比对，识别潜在的不一致描述。检测过程不仅判断是否存在冲突，还会对冲突进行类型分类，并结合上下文信息评估冲突的严重程度，从而为用户提供更具决策价值的分析结果。
- **溯源验证层**：对于检测到的冲突，系统支持进入可选的溯源验证流程。该阶段会结合外部信息检索或上下文证据，对冲突事实进行进一步校验，并在此基础上生成可解释的修正建议，帮助用户判断应统一或修订的事实版本。
- **结果持久化与推送**：分析完成后，系统会将最终结果写入 PostgreSQL 进行持久化存储，同时将高频访问的数据缓存在 Redis 中。通过 WebSocket，分析进度与冲突结果会被实时推送至前端界面，用户可以在可视化视图中即时查看分析结果、冲突分布和文档整体一致性状况。

通过这一完整的数据流设计，PatPat-Inconsistency-Hunter 将复杂的长文档分析过程拆解为清晰、可控的阶段，使事实一致性检测从不可见的后台计算，转变为用户可理解、可追溯的分析过程。

三、技术栈

3.1 后端技术栈

后端系统围绕“长文档处理、状态复杂、分析链路可控、协作实时性高”等核心需求进行技术选型。整体设计优先考虑异步能力、可扩展性以及复杂分析流程的支撑能力，而非仅满足基础 CRUD 场景。通过将 Web 接入、智能分析、状态管理与数据持久化进行合理分层，后端能够在保证性能的同时，支持多阶段、可回溯的文档一致性分析流程。

在技术选型上，本系统以 Python 异步生态为核心，结合大语言模型与视觉模型能力，并通过缓存与数据库的组合，兼顾实时性与数据可靠性。

下表列出了后端所使用的主要技术组件及其在系统中的职责：

类别	技术	版本	用途
Web框架	FastAPI	0.115	异步 API 服务、WebSocket 实时通信
ORM	SQLAlchemy	2.0	异步数据库操作与模型映射
LLM	DeepSeek API	-	事实提取、冲突检测、溯源验证
视觉模型	InternVL3_5-4B-HF	-	文档图片与图表内容识别
缓存	Redis	7	状态管理、事实黑板、分布式锁
数据库	PostgreSQL	15	文档、分析结果与协作数据持久化
认证	python-jose + bcrypt	-	JWT 认证与密码加密
文档解析	PyMuPDF/python-docx	-	PDF/DOCX解析
导出	WeasyPrint/ReportLab	-	分析报告与文档导出

3.1.1 技术选型说明

- **FastAPI 与异步架构**：FastAPI 作为后端核心 Web 框架，提供高性能的异步请求处理能力，并天然支持 WebSocket，用于协作编辑、任务进度和冲突结果的实时推送。配合 Python 的异步生态，能够有效支撑长时间运行的分析任务与高并发状态查询。
- **SQLAlchemy 2.0 与数据模型设计**：本系统采用 SQLAlchemy 2.0 的异步 ORM 模式，对数据库访问进行统一抽象，既保证了数据一致性，又避免了业务逻辑与底层 SQL 强耦合。分析结果、文档版本、协作历史等核心数据均通过 ORM 模型进行管理，便于后续扩展和维护。
- **大语言模型与视觉模型协同**：DeepSeek API 负责文本级的理解与推理任务，包括事实抽取、冲突判断与修正建议生成；InternVL3_5-4B-HF 以本地部署方式运行，用于解析文档中的图片和图表内容。二者共同构成多模态分析能力，使系统能够覆盖“图文混合”的长文档场景。
- **Redis 与 PostgreSQL 的分工协作**：Redis 被设计为统一的实时状态与中间结果中心，承载任务状态、分析进度、事实黑板、冲突黑板以及分布式锁等高频数据；PostgreSQL 则用于长期可靠的数据存储，负责用户信息、文档内容、分析结果和统计数据的持久化。二者分工明确，兼顾性能与稳定性。
- **文档解析与结果导出能力**：系统针对长文档场景引入多种解析与导出工具，支持从 PDF、DOCX 等格式中提取内容，并将分析结果以 PDF、DOCX 等形式导出，满足审核、评估和交付等实际使用需求。

通过上述后端技术栈的组合，PatPat-Inconsistency-Hunter 能够在复杂分析逻辑与实时协作并存的场景下保持良好的性能与可维护性，为长文档事实一致性检测提供稳定可靠的基础支撑。

3.2 前端技术栈

前端系统的设计目标并不仅是展示结果，而是作为长文档分析与协作流程中的核心交互载体。用户需要在前端完成文档编辑、多人协作、冲突定位、分析结果浏览以及统计指标查看等操作，这要求前端具备良好的组件化能力、实时响应能力以及对复杂状态的可视化表达能力。

基于上述需求，本目前端技术选型以现代化的 React 生态为核心，兼顾开发效率、交互体验与可维护性，并与后端的实时通信和分析流程形成紧密配合。

下表列出了前端所使用的主要技术组件及其用途：

类别	技术	版本	用途
框架	React	18	构建用户界面与状态驱动视图
构建工具	Vite	5	快速开发与高效构建
样式	TailwindCSS	3	原子化 CSS 与统一视觉风格
编辑器	TipTap	2.1	富文本编辑与结构化内容管理
动画	Framer Motion	-	页面与交互动画效果
图表	Recharts	-	分析结果与统计数据可视化
请求	Axios	-	与后端 API 的通信

3.2.1 技术选型说明

- React 18 与组件化架构**：前端基于 React 18 构建，采用组件化与状态驱动的开发模式，将文档编辑、协作视图、分析结果展示和仪表盘等功能拆分为独立模块。通过合理的状态管理和组件组合，前端能够在复杂交互场景下保持清晰的数据流和良好的可维护性。
- Vite 提升开发与构建效率**：Vite 作为构建工具，为项目提供了快速的冷启动和即时的热更新能力，显著提升开发效率。在构建阶段，Vite 能够生成体积更小、加载更快的前端资源，适合文档编辑与数据可视化并存的应用场景。
- TailwindCSS 与统一视觉体系**：前端样式采用 TailwindCSS 的原子化方案，通过预定义的样式组合快速构建界面，同时保持整体风格的一致性。该方式减少了样式文件的复杂度，使界面设计能够随着功能迭代而灵活调整。
- TipTap 富文本编辑能力**：TipTap 作为核心编辑器，承担长文档编辑与结构化内容管理的任务。其基于 ProseMirror 的架构，支持对文档内容进行精细化控制，为后端分析流程提供稳定、可解析的文档结构，同时也为多人协作和冲突定位提供基础。
- Framer Motion 与交互反馈**：在分析过程、状态切换和结果展示中，引入 Framer Motion 提供平滑的动画效果，使用户能够直观感知任务进度、视图变化和交互反馈，降低复杂功能带来的认知负担。
- Recharts 与结果可视化**：分析结果和统计指标通过 Recharts 进行可视化展示，包括冲突类型分布、严重度趋势和任务统计等信息，帮助用户从整体层面快速理解文档的一致性状况。
- Axios 与后端通信**：前端通过 Axios 与后端 API 进行通信，统一处理请求、响应与异常情况，并与 WebSocket 通道配合，实现实时协作与分析结果的即时更新。

通过上述前端技术栈的组合，系统能够在支持复杂长文档编辑与实时协作的同时，为用户提供清晰、稳定且易于理解的交互体验。

3.3 云原生部署

为降低部署复杂度、提升环境一致性，并满足分析任务与协作服务并存的运行需求，本系统在部署层面采用基于 Docker Compose 的云原生部署方案。系统通过容器化方式对前端、后端、缓存和数据库进行统一编排，使整套系统能够以“一键启动”的方式在本地或服务器环境中运行。

该部署方式特别适用于包含长时间分析任务、实时协作通信以及多种存储组件的应用场景，既便于开发调试，也为后续迁移到更复杂的容器编排平台预留了空间。

3.3.1 服务架构划分

系统整体由四类核心服务组成，各服务之间通过容器网络进行通信，职责边界清晰、相互解耦：

- **前端服务 (client)**
前端服务基于 Nginx 运行，负责托管 React 构建后的静态资源，并作为用户访问系统的入口。通过 Nginx 的反向代理能力，前端可以无感知地与后端 API 和 WebSocket 服务进行交互。
- **后端服务 (server)**
后端服务基于 FastAPI 与 Uvicorn 运行，承载文档分析、协作管理、事实提取、冲突检测等核心业务逻辑。该服务同时支持 HTTP API 与 WebSocket 通信，并可配置 GPU 资源，用于加速本地部署的视觉模型 (InternVL3.5) 推理过程。
- **缓存服务 (redis)**
Redis 作为统一的实时状态中心运行于独立容器中，负责任务状态、分析进度、事实黑板、冲突黑板以及分布式锁等高频数据的管理，为系统提供低延迟的状态同步能力。
- **数据库服务 (postgres)**
PostgreSQL 负责系统的持久化存储，包括用户信息、文档内容、分析结果、协作房间与历史记录等数据。数据库容器通过端口映射暴露给宿主机，便于本地调试与数据管理。

3.3.2 Docker Compose 部署结构

```
services:
  client:          # 前端服务 (Nginx + React)
    port: 3000

  server:          # 后端服务 (FastAPI + Uvicorn)
    port: 8000
    gpu: nvidia    # GPU 加速视觉模型推理

  redis:           # 缓存与状态中心
    port: 6379

  postgres:        # 数据持久化存储
    port: 5433
```

3.3.3 部署设计特点

该云原生部署方案具备以下特点：

设计目标	对应方案	实际收益
服务解耦	前端、后端、缓存与数据库分别以独立容器运行	避免环境依赖冲突，降低单点故障影响，提升系统整体稳定性
环境一致性	通过 Docker 镜像封装运行环境并统一编排	确保开发、测试与部署环境行为一致，减少环境差异导致的问题
弹性扩展	后端服务可按需独立扩容，缓存与数据库支持替换为高可用部署	能够应对分析负载波动，为后续规模化部署预留空间
硬件加速	后端容器支持 GPU 资源挂载，用于视觉模型推理	在需要时显著提升图片与图表解析性能，降低分析耗时
快速启动	使用 Docker Compose 进行一键编排与启动	开发者和评审人员可在最少配置成本下运行完整系统

通过上述云原生部署设计，PatPat-Inconsistency-Hunter 能够在保证系统复杂功能的同时，保持良好的部署体验和运行稳定性，为长文档事实一致性分析提供可靠的基础运行环境。

四、核心功能实现

4.1 事实提取

事实提取是系统进行一致性分析的第一步，也是后续冲突检测与溯源校验的基础能力。系统在完成文档解析与多模态理解后，会对全文内容进行统一扫描，从中自动抽取可被独立校验和比对的“原子事实”，并以结构化形式写入事实黑板，为全局分析提供可靠输入。

在设计上，事实提取并不追求对全文语义的完全复述，而是聚焦于**对一致性最敏感、最容易产生冲突的关键信息**。这些信息一旦在不同章节出现偏差，往往会直接影响文档的可信度和逻辑完整性。

本系统目前支持自动抽取以下六类原子事实：

事实类型	标识	说明	示例
数值类	numerical	具体数字、比例、金额等量化信息	“项目预算为 100 万元”
时间类	temporal	日期、时间点或时间区间	“项目于 2024 年 1 月启动”
实体类	entity	人名、组织、机构、地点等实体	“项目负责人是张三”
定义类	definition	对概念、术语的定义性描述	“AI 是模拟人类智能的技术”
结论类	conclusion	研究结论或判断性表述	“实验结果表明方案可行”

事实类型	标识	说明	示例
关系类	relationship	实体之间的明确关系	"A 公司收购了 B 公司"

4.1.1 多模态事实的统一表示

在包含图片或图表的文档中，系统会先通过视觉模型生成图片的语义描述，再将其与文本内容合并进入同一分析流程。对于来源于图片的事实，系统在类型标识前增加 `image_` 前缀，以明确其来源，例如：

- `image_numerical`：从图表中识别出的数值信息
- `image_temporal`：图片中标注的时间信息

这种区分方式使系统在后续冲突检测与溯源阶段，能够明确事实的来源位置（文本或图片），并在图文不一致的场景下进行针对性分析。

4.1.2 事实提取结果的结构化管理

所有提取出的事实都会被标准化为统一的数据结构，并登记到事实黑板中。每条事实不仅包含其类型和内容，还会保留对应的原文片段、所在章节及位置信息，为后续的冲突定位、可视化高亮和人工审阅提供精确依据。

通过将长文档中的关键信息拆解为可管理、可比对的原子事实，本系统为一致性检测建立了清晰、可扩展的基础，使复杂文档的全局事实校验成为可能。

4.2 冲突检测

在完成事实提取并构建事实黑板之后，系统进入冲突检测阶段。该阶段的核心目标是：**在全文范围内对同类事实进行系统性比对，识别前后不一致、相互矛盾或语义冲突的描述**，并将结果结构化输出，供后续溯源验证与人工修订使用。

与传统基于关键词或相邻文本比对的方式不同，系统的冲突检测建立在“事实级别”的统一表示之上。所有检测逻辑均围绕已抽取的原子事实展开，从而能够覆盖跨章节、跨段落、跨图文的隐性冲突，而不仅限于表层文本差异。

4.2.1 支持的冲突类型

系统目前支持检测以下 **八种冲突类型**，覆盖长文档中最常见、也最具风险的一致性问题：

冲突类型	描述	示例
数值冲突	同一概念在不同位置出现不同的数值	"项目预算为 100 万元" vs "项目预算为 150 万元"
时间冲突	同一事件在不同位置出现不同的时间描述	"会议于 2024 年 1 月举行" vs "会议于 2024 年 2 月举行"
实体冲突	同一概念指向不同的实体或人物	"项目负责人是张三" vs "项目负责人是李四"

冲突类型	描述	示例
类别冲突	同一概念被归入不同的分类体系	"机器学习分为监督学习和无监督学习" vs "机器学习分为深度学习和强化学习"
定义冲突	同一术语在不同位置具有不同定义	"AI 是模拟人类智能的技术" vs "AI 是自动化决策系统"
逻辑冲突	不同陈述之间在逻辑上无法同时成立	"所有用户都已登录" vs "部分用户未登录"
空间冲突	同一对象在不同位置出现不同的空间描述	"办公室位于 3 楼" vs "办公室位于 5 楼"
图文冲突	文本描述与图片或图表内容不一致	文本描述"销售额增长 20%" vs 图表显示"销售额下降 10%"

通过将冲突类型进行明确分类，系统不仅能够告诉用户“是否存在冲突”，还能够说明“冲突发生在什么层面”，从而显著降低人工理解和处理的成本。

4.2.2 严重度评估机制

除了冲突类型判定之外，系统还会对每一条冲突进行严重度评估，用于衡量其对文档整体可信度和可读性的影响程度。严重度以连续分值形式计算，并映射为四个等级：

级别	分数范围	颜色标识	含义
轻微	0.0-0.3	● 绿色	表述存在差异，但不影响核心结论
中等	0.3-0.6	● 黄色	需要关注和核实
严重	0.6-0.9	● 橙色	可能导致理解偏差，建议修正
矛盾	0.9-1.0	● 红色	明确冲突，必须修正

严重度评估综合考虑事实差异幅度、上下文语义、事实类型以及其在文档结构中的重要性，使冲突结果不仅“可见”，而且“可排序、可决策”。

4.2.3 冲突检测的工程实现思路

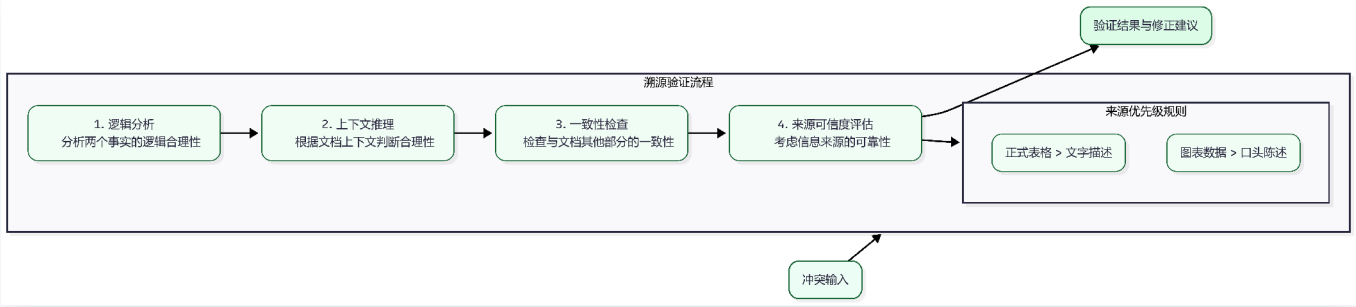
在实现层面，系统会基于事实类型对候选事实进行分组，并生成潜在冲突对；随后通过规则判断与语言模型推理相结合的方式，对事实对进行一致性分析。对于来源不同（如文本与图片）的事实，系统会自动进入图文一致性检测逻辑，避免将多模态冲突遗漏在分析之外。

所有检测结果都会被写入冲突黑板，并保留对应的事实来源、原文位置与类型标签，支持在前端进行高亮定位、分类筛选和统计分析，为后续的溯源验证与人工修订提供清晰依据。

4.3 溯源校验

在冲突检测之后，系统提供可选的溯源校验流程，用于对已发现的冲突进行二次验证，进一步判断“哪一种表述更可能正确”，并给出可执行的修正建议。该能力的目标不是简单复述冲突，而是尽可能把冲突从“发现问题”推进到“解决问题”：为人工审阅提供更明确的决策依据与修改方向，从而降低反复核对和沟通成本。

溯源校验的输入是一条冲突记录（包含冲突双方的事实内容、类型、位置与上下文片段）。系统会围绕冲突双方分别构建证据，并按照一套可解释的验证流程进行综合评估，最终输出验证结论与建议统一口径。



4.3.1 验证维度说明

- **逻辑分析**：系统首先从常识与逻辑约束出发，判断冲突双方是否存在明显的不合理性，例如数值规模是否离谱、时间顺序是否自相矛盾、因果关系是否成立等。该步骤主要用于快速排除“显然错误”的候选事实。
- **上下文推理**：系统会回到冲突事实所在的上下文，结合段落语义、章节主题和表述意图判断其合理性。例如同一术语在定义章节与讨论章节出现差异时，会倾向于以定义章节的表述为主；同一指标在“摘要”与“实验结果”出现冲突时，会进一步结合结果章节的细节描述推断更可信的一方。
- **一致性检查**：除冲突双方之外，系统还会检索文档其他位置是否出现第三方证据（例如同一指标在多个章节重复出现的口径）。当某一口径与全文其他位置保持一致，而另一口径仅在局部出现时，系统会倾向于选择更具全局一致性的事实版本，并标记另一处为需要统一的异常点。
- **来源可信度评估**：对于图文混合的长文档，事实来源本身就是重要信号。系统会对事实来源进行权重判断，例如优先级通常符合以下经验规则：
 1. 正式表格优于散落的文字描述
 2. 图表中明确标注的数据优于叙述性表达
 3. 结论章节的汇总性表述需要回溯到方法与结果章节交叉验证

当冲突涉及图片来源事实时，系统能够利用“图片描述注入”后的证据链，将图表数据与文字陈述放在同一尺度下进行比较，从而更准确地识别图文不一致与口径漂移。

4.3.2 输出结果形式

溯源校验完成后，系统会输出两类信息：

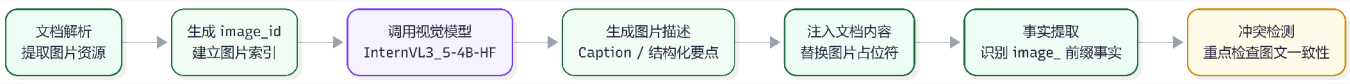
- **验证结论**：更可能正确的事实口径、置信度或倾向性判断，以及关键证据摘要
- **修正建议**：建议统一为哪个口径、需要修改的段落位置、推荐的替换表述方向

该阶段的结果会与冲突记录一起进入后续的报告生成与可视化展示流程，使用户能够在界面中直接查看“冲突是什么、为什么更倾向某一方、应该怎么改”，从而将一致性治理落到可执行层面。

4.4 图片内容识别

为避免长文档分析只覆盖纯文本而忽略图表、表格等关键证据，系统引入本地部署的 InternVL3_5-4B-HF 视觉模型，对文档中的图片内容进行语义理解与结构化补全。该能力会在文档解析阶段抽取图片资源，并在分析前将图片信息“转写”为可计算的文本描述注入到文档内容中，使后续的事实提取与冲突检测能够把图文信息放在同一尺度下进行一致性校验。

从功能链路上看，图片识别并不是孤立的“看图”能力，而是服务于一致性检测的多模态数据补全：图片中的数值、趋势、实体关系会被转化为可追踪的文本证据，并在事实层面保留来源标记，从而支持图文冲突检测与结果解释。



4.4.1 优势

- 本地部署，视觉推理过程不依赖外部网络，适合在受限环境或离线场景运行
- 能覆盖图表、表格、流程图等常见文档图片类型，将视觉信息转化为可参与分析的文本证据
- 对图片中的关键要素具备较强的提取能力，包括数值信息、趋势变化、实体关系与结构化要点，从而提升图文一致性检测的召回与解释性

4.5 协作编辑

在长文档的一致性治理场景中，协作编辑不仅是“多人同时写文档”，更涉及权限控制、状态同步以及一致性风险的实时暴露。系统在设计协作能力时，将“高效协作”和“事实一致性保障”作为同等重要的目标，通过明确的协作模式和实时通信机制，避免多人写作过程中事实口径失控的问题。

系统目前支持两种协作模式，适配不同类型的写作与审核场景：

模式	特点	适用场景
实时协同	多人可同时编辑同一文档内容，修改即时同步	头脑风暴、初稿撰写、快速迭代
分章节锁定	房主分配章节编辑权限，每人负责固定章节	分工明确的论文、报告、技术文档

在实时协同模式下，系统更强调编辑流畅性和即时反馈，适合早期内容生成与快速讨论；而在分章节锁定模式下，通过章节级权限控制，避免多人同时修改同一内容导致的冲突，适合结构稳定、需要严格口径控制的长文档写作阶段。

4.5.1 协作功能说明

围绕上述两种协作模式，系统提供了一组完整的协作支撑功能，确保多人写作过程可控、可追溯：

- **协作房间管理**：支持创建、加入和结束协作房间，并通过邀请码机制控制成员访问范围，确保协作环境的安全性与独立性。
- **实时内容同步**：基于 WebSocket 实现文档内容的实时同步，编辑操作能够在协作者之间即时广播，保证各方看到的始终是同一版本内容。
- **协作者状态可视化**：系统会同步协作者的光标位置与编辑状态，使参与者能够直观感知他人的编辑范围，减少误操作和重复修改。
- **章节级编辑锁定**：在分章节锁定模式下，系统通过章节锁机制限制编辑权限，确保每个章节在任意时刻只被授权人员修改，从机制上避免内容覆盖与责任不清的问题。
- **实时冲突检测与结果推送**：在协作过程中，系统可在保存或显式触发时对当前文档状态进行冲突检测，并将结果通过 WebSocket 推送给相关协作者或房主，使一致性问题尽早暴露，而不是在最终合并阶段集中出现。
- **协作成果导出**：支持将协作完成的文档按多种格式导出，并可选择是否附带事实分析与冲突检测结果，满足审核、评估和交付等不同使用需求。

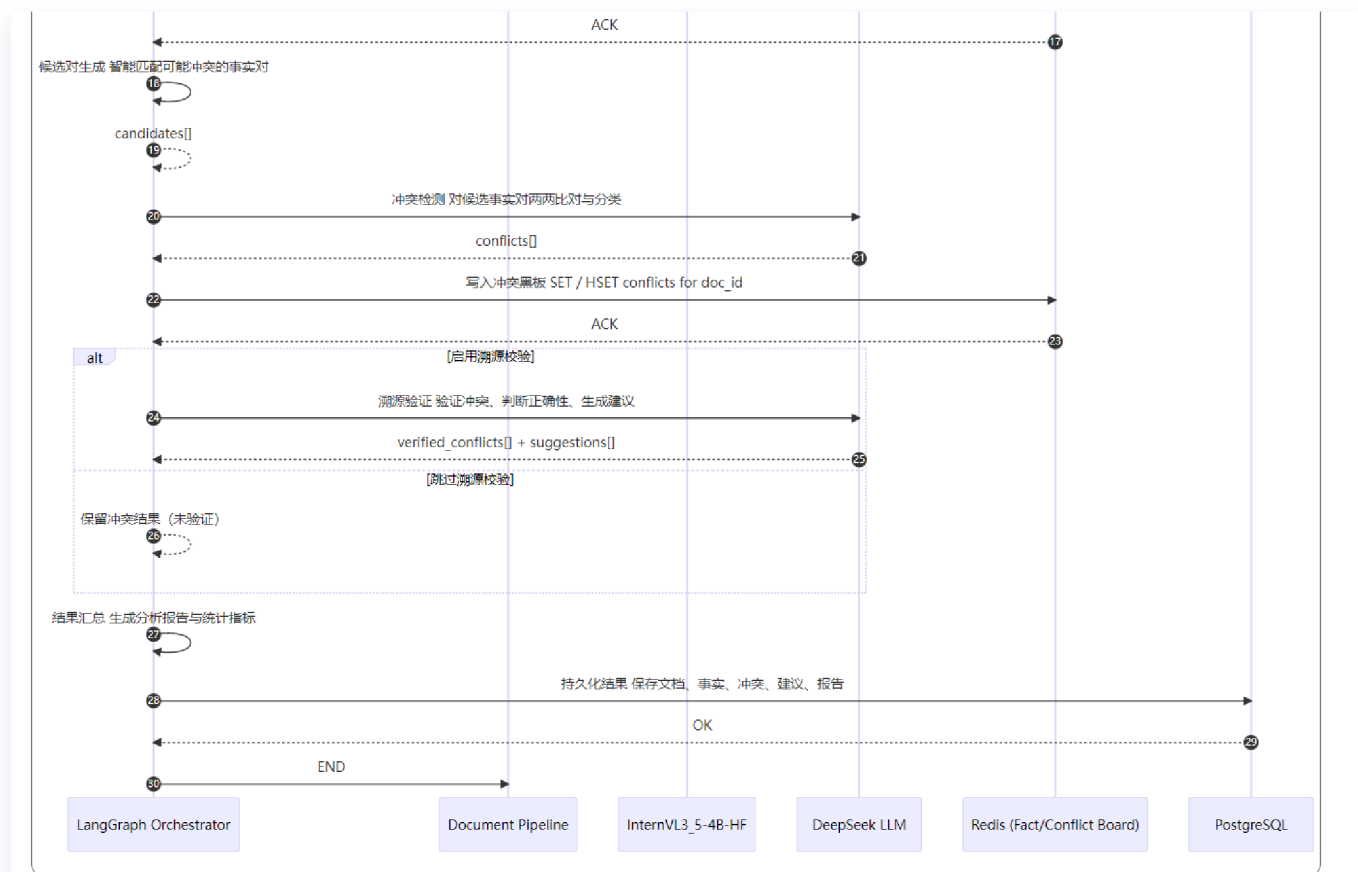
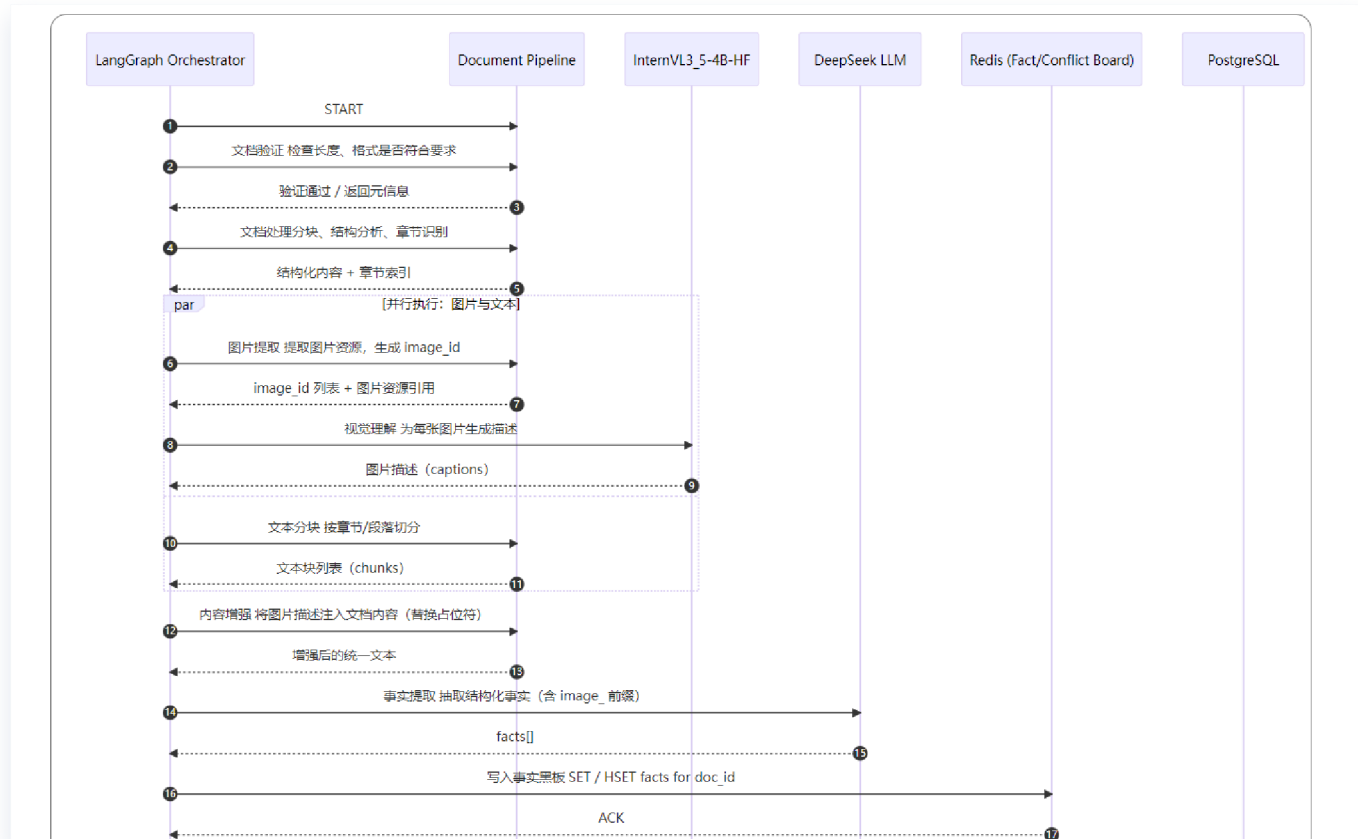
通过将协作控制、实时通信与一致性检测深度结合，系统将多人写作从“事后对齐口径”的被动模式，转变为“写作过程中持续约束事实”的主动模式，为长文档协作提供更可靠的工程化保障。

五、LangGraph 智能体设计

5.1 workflow 架构

本系统使用 **LangGraph** 将“长文档一致性分析”组织为一张可编排的图结构工作流。与线性脚本不同，LangGraph 的优势在于：每个节点职责清晰、可独立观测与重试；同时支持并行分支（例如图片与文本处理并行），以及条件分支（是否启用溯源校验），使分析链路既具备工程可控性，也具备扩展空间。

下面的工作流图展示了从输入文档到输出分析报告的完整路径：先进行文档验证与结构化处理，再并行处理图片与文本，随后完成内容增强、事实提取、候选对生成与冲突检测；最后根据配置选择是否进入溯源校验，最终汇总结果并持久化。



5.2 节点定义与状态流转

本节从 **workflow 执行层**的角度，说明系统内部是如何通过一组明确的节点定义与统一的状态对象，将长文档一致性分析过程组织为一个**可观测、可回滚、可扩展**的状态机流程。

整个分析流程基于 LangGraph 构建，核心思想是：**分析过程不依赖隐式上下文，而是围绕一个显式的状态对象逐步推进**。每一个节点只关心“从状态中读取什么、向状态中写回什么”，从而保证流程在复杂分支与失败场景下依然具备良好的可控性。

5.2.1 状态定义

系统使用统一的 `AnalysisState` 作为 workflow 中唯一的状态载体。该状态既包含任务输入与最终输出，也覆盖了中间产物与运行元信息，用于支撑进度推送、失败定位与重试恢复等能力。

```
class AnalysisState(TypedDict):
    """分析状态 - LangGraph 工作流的状态"""
    # 输入
    document: DocumentInput      # 原始文档
    task_id: str                  # 任务ID
    skip_verification: bool       # 是否跳过验证
    content_json: Dict[str, Any]  # 富文本JSON

    # 中间状态
    chunks: List[DocumentChunk]   # 文档分块
    images: List[ImageInfo]        # 图片信息
    facts: List[FactWithSource]    # 提取的事实
    candidate_pairs: List[tuple]   # 候选冲突对
    conflicts: List[ConflictPair]  # 检测到的冲突

    # 元信息
    current_step: str              # 当前步骤
    progress: float                # 进度 (0-100)
    error: Optional[str]           # 错误信息

    # 输出
    result: Optional[AnalysisResult] # 分析结果
```

从设计上看，`AnalysisState` 体现了三个层次的状态语义：

- 1. 输入与配置**，用于保证任务具备可重放性；
- 2. 分析中间结果**，承载文档解析、事实建模与冲突推理的阶段性产物；
- 3. 运行元信息**，用于描述当前执行到哪一步、整体进度以及是否发生异常。

这种统一状态模型，使得分析流程不再依赖隐式调用顺序，而是以“状态演进”的方式自然展开。

5.2.2 节点定义：

在状态模型之上，系统将整个分析过程拆解为一组职责清晰的节点。每个节点都对应分析流水线中的一个关键阶段，并遵循一致的输入 / 输出约定：节点从 `AnalysisState` 中读取所需字段，完成处理后，仅将自己的结果写回状态。

整体节点划分如下：

节点	功能	输入	输出
<code>validate_document</code>	验证文档格式和长度	<code>document</code>	<code>validation_result</code>
<code>process_document</code>	分块和结构分析	<code>document</code>	<code>chunks, metadata</code>
<code>extract_images</code>	提取图片信息	<code>content_json</code>	<code>images</code>
<code>enhance_content</code>	注入图片描述	<code>chunks, images</code>	<code>enhanced_chunks</code>
<code>extract_facts</code>	事实提取	<code>enhanced_chunks</code>	<code>facts</code>
<code>generate_pairs</code>	生成候选冲突对	<code>facts</code>	<code>candidate_pairs</code>
<code>detect_conflicts</code>	冲突检测	<code>candidate_pairs</code>	<code>conflicts</code>
<code>verify_conflicts</code>	溯源验证	<code>conflicts</code>	<code>verified_conflicts</code>
<code>summarize_results</code>	结果汇总	<code>all</code>	<code>result</code>

这种节点拆分方式的核心目的并不是追求“步骤多”，而是将**高成本、不确定性较高的分析逻辑隔离在独立节点中**，从而为后续的并行化、跳过执行或失败重试提供结构基础。

5.2.3 状态流转与执行逻辑

在运行时，LangGraph 会将上述节点组织为一条有向的状态流转路径。分析任务从文档校验开始，依次完成结构解析、多模态增强、事实建模与一致性推理，最终生成可视化与可解释的分析结果。

其中，冲突溯源验证被设计为一个**可选分支**。当 `skip_verification` 为真时，流程在完成冲突检测后直接进入结果汇总阶段；当开启验证时，则会在冲突检测后追加溯源分析节点，用于进一步提升结果可信度。这种分支能力使系统能够在“速度优先”和“严谨优先”两种模式之间灵活切换。

在整个执行过程中，`current_step` 与 `progress` 会随着节点推进持续更新，并同步写入实时状态存储，用于前端进度展示与协作方订阅。这种显式的阶段标记机制，使得长文档分析不再是一个“黑盒任务”，而是一个对用户与系统都可感知的渐进过程。

当任一节点发生异常时，错误信息会被记录在状态中，工作流可在该节点终止，或在人工/系统干预后从指定阶段重新执行。借助 LangGraph 的状态化执行模型，分析流程天然具备节点级重试与回退能力，为复杂、多阶段推理任务提供了工程上的稳定性保障。

5.3 容错与回滚机制

本节围绕系统在“LLM 输出不确定 + 长耗时链路 + 多节点编排”的场景下的稳定性要求，说明我们在输出校验、调用保护、节点级失败隔离与可观测性方面的容错设计。

5.3.1 LLM 输出校验（结构化输出 + JSON 修复）

为降低生成式模型在复杂指令下出现的“格式漂移”，系统对关键输出采用“结构化约束 + Schema 校验 + 修复兜底”的组合策略。下列代码片段展示了后端在调用 LLM 时对输出结构进行约束与验证的核心做法：

```
response = await self.chat(
    messages=messages,
    response_format={"type": "json_object"},
)
try:
    return json.loads(response)
except json.JSONDecodeError:
    return self._repair_json(response)

schema = response_model.model_json_schema()
json_response = await self.chat_with_json(messages=modified_messages)
return response_model.model_validate(json_response)
```

- 1.JSON 模式：** `chat_with_json(..., response_format={"type": "json_object"})` 约束模型输出为 JSON 对象。
- 2.Schema 注入与校验：** `chat_structured(..., response_model=...)` 将 `response_model.model_json_schema()` 注入提示词，并在返回后执行 `response_model.model_validate(json_response)`。
- 3.JSON 修复兜底：** 当 `json.loads` 失败时，先用正则抽取 JSON 片段，再使用 `json_repair.repair_json` 修复（`DeepSeekClient._repair_json`）。

通过在“输出侧”引入严格的结构约束与工程化兜底，系统能够显著降低因输出不可解析导致的链路中断，为后续事实抽取与冲突检测提供稳定输入。

5.3.2 重试、退避与超时保护

针对外部模型调用与网络波动带来的不确定性，系统在调用层引入超时控制，并在通用异步任务层引入指数退避重试，确保“短时抖动不致失败，持续失败可快速暴露”。对应的关键实现如下：

```

client = DeepSeekClient(timeout=120)

for attempt in range(max_retries + 1):
    try:
        return await async_function(*args, **kwargs)
    except Exception as e:
        if attempt < max_retries:
            await asyncio.sleep(current_delay)
            current_delay *= backoff

```

1. DeepSeek 调用超时： `DeepSeekClient(timeout=120)` （ `llm_client.py` ）。
2. 通用异步重试： `server/app/utils/async_utils.py::retry_async` （指数退避，默认最多 3 次）。

在事实抽取、冲突检测等批量调用中，系统结合超时、重试与并发限流，避免因单次调用异常触发整任务失败；当重试耗尽仍失败时，错误会被记录并反馈到任务状态中，便于前端提示与后续排障。

5.3.3 节点级失败隔离与降级输出（软回滚）

为避免“全有或全无”的不可交付结果，LangGraph 工作流在节点粒度实现失败隔离：某一节点失败时，通过降级输出继续推进后续节点，并保留已成功产出。下列片段展示了节点异常捕获与跳过验证时的结果保留策略。

```

try:
    image_result = await self._node_extract_images(state)
except Exception:
    image_result = {"images": [], "current_step": "图片处理失败", "progress": 12}

if not conflicts or skip_verification:
    conflict_reports = [
        ConflictReport(conflict=c, verification=None)
        for c in conflicts
    ]

```

LangGraph 将分析过程拆成多个节点，当部分节点失败时：

1. **图片识别失败**：跳过图片增强，仍继续进行纯文本事实提取与冲突检测。
2. **验证失败或跳过验证**：保留冲突检测结果，但标记为“未验证”，并在前端提示用户人工复核。
3. **部分结果保留**：已经成功生成的 `facts` 与 `conflicts` 会被保留并用于后续导出/持久化，避免“全有或全无”。

该设计使系统能够在真实环境下以“可用优先”的方式交付结果，同时把不可用部分明确收敛到可解释的失败节点。

5.3.4 可观测性：进度推送与任务状态一致性

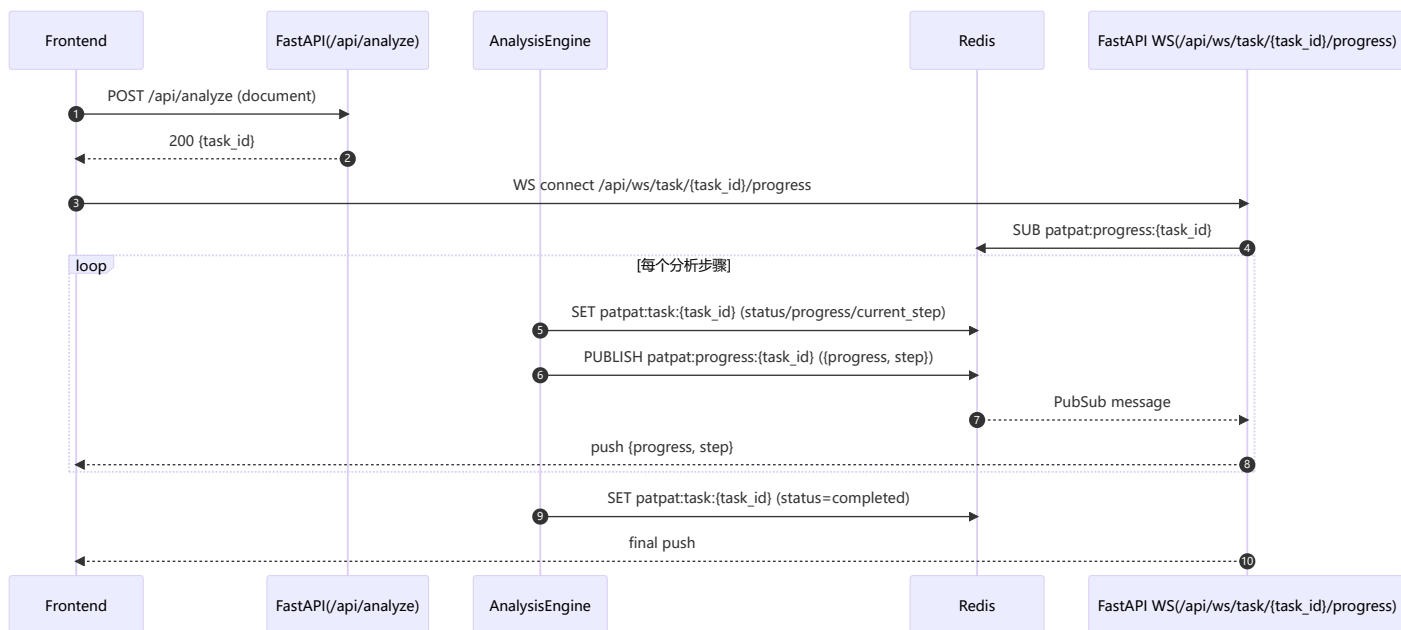
由于分析任务通常为长耗时异步执行，若缺少过程可观测性，前端体验与调试效率都会显著下降。因此系统采用“Redis 持久化任务态 + PubSub 推送进度事件 + WebSocket 实时转发”的组合机制，确保每个步骤的状态可追踪、可回放：

```
# Redis: 更新任务状态 + 发布进度事件
task_data = await self.get_task(task_id)
if task_data:
    task_data["progress"] = progress
    task_data["current_step"] = step
    await self.save_task(task_id, task_data)
await self.publish_progress(task_id, progress, step)

# FastAPI: WebSocket 转发 Redis PubSub 进度消息
@router.websocket("/ws/task/{task_id}/progress")
async def websocket_progress(websocket: WebSocket, task_id: str):
    await websocket.accept()
    pubsub = await redis_client.subscribe_progress(task_id)
    ...
    await websocket.send_json(data)
```

1. **进度发布**： `RedisClient.update_task_progress(...)` 会同时更新任务状态并通过 `publish_progress(...)` 向频道 `patpat:progress:{task_id}` 发布 `{progress, step}`。
2. **WebSocket 转发**：后端 WebSocket `/api/ws/task/{task_id}/progress` 订阅 Redis PubSub，并将进度实时推送给前端（进度条/步骤文案）。

该设计保证：即使任务失败，前端也能获得明确失败阶段（ `current_step` ）和错误信息（ `error` ），提升可调试性与用户体验。



六、Prompt 工程

本章从工程实现角度说明系统 Prompt 的组织方式与关键约束。我们将 LLM 能力拆解为“事实提取 → 冲突检测 → 溯源验证”三类 Prompt，并通过结构化输出与工程侧校验保证输出可解析、可追溯、可复核。

6.1 事实提取 Prompt

实现中对应变量为：`FACT_EXTRACTION_SYSTEM_PROMPT`。为保证后续冲突检测能够稳定对齐字段含义，该 Prompt 强制采用结构化输出，并对事实的“证据锚定”提出显式要求：

- (1) 输出必须为结构化 JSON（工程侧会对结构进行校验与必要修复）。

(2) 仅抽取原文可验证事实：不补全、不猜测，要求给出可回溯的 `source_text` 。

(3) 图片内容纳入抽取：对 `[图片内容开始: ...] ... [图片内容结束]` 的描述同样抽取事实，并在 `fact_type` 前加 `image_` 前缀，便于后续图文一致性检测。

系统定义的事实类型共 6 类（与候选对生成策略相匹配）：`numerical`（数值）、`temporal`（时间）、`entity`（实体）、`definition`（定义）、`conclusion`（结论）、`relationship`（关系）。

6.2 冲突检测 Prompt

对应变量：`CONFLICT_DETECTION_SYSTEM_PROMPT` 。

核心目标是判断两条事实是否冲突，并输出可直接用于前端展示与排序的结构化字段：

- 1. `conflict_type`：冲突类型。
- 2. `severity`：严重程度（用于排序/筛选）。
- 3. `description/suggestion`：冲突解释与修正建议。

冲突类型集合：`numerical / temporal / entity / categorical / definition / logical / spatial` 。

对图文冲突的特殊处理：当某条事实 `fact_type` 以 `image_` 开头时，Prompt 会要求重点比对“图表数据/日期/趋势”与文本描述是否一致。

6.3 溯源验证 Prompt

对应变量：`FACT_VERIFICATION_SYSTEM_PROMPT` 。

溯源验证用于对冲突进行二次复核（可选阶段）：

- 1. 结合上下文进行推理（而非只看两条事实文本）。
- 2. 引入来源可靠性与图文提示信息，尽量判断“冲突是否成立、哪条更可信”。
- 3. 输出可执行的修正建议，并给出置信度，便于前端提示“需要人工复核”的冲突。

6.4 幻觉防护策略

针对 LLM 可能产生的幻觉问题，系统采用多层防护。为便于从工程实现角度理解各层策略的作用边界，下表对主要防护手段、落地方式与预期效果进行归纳。

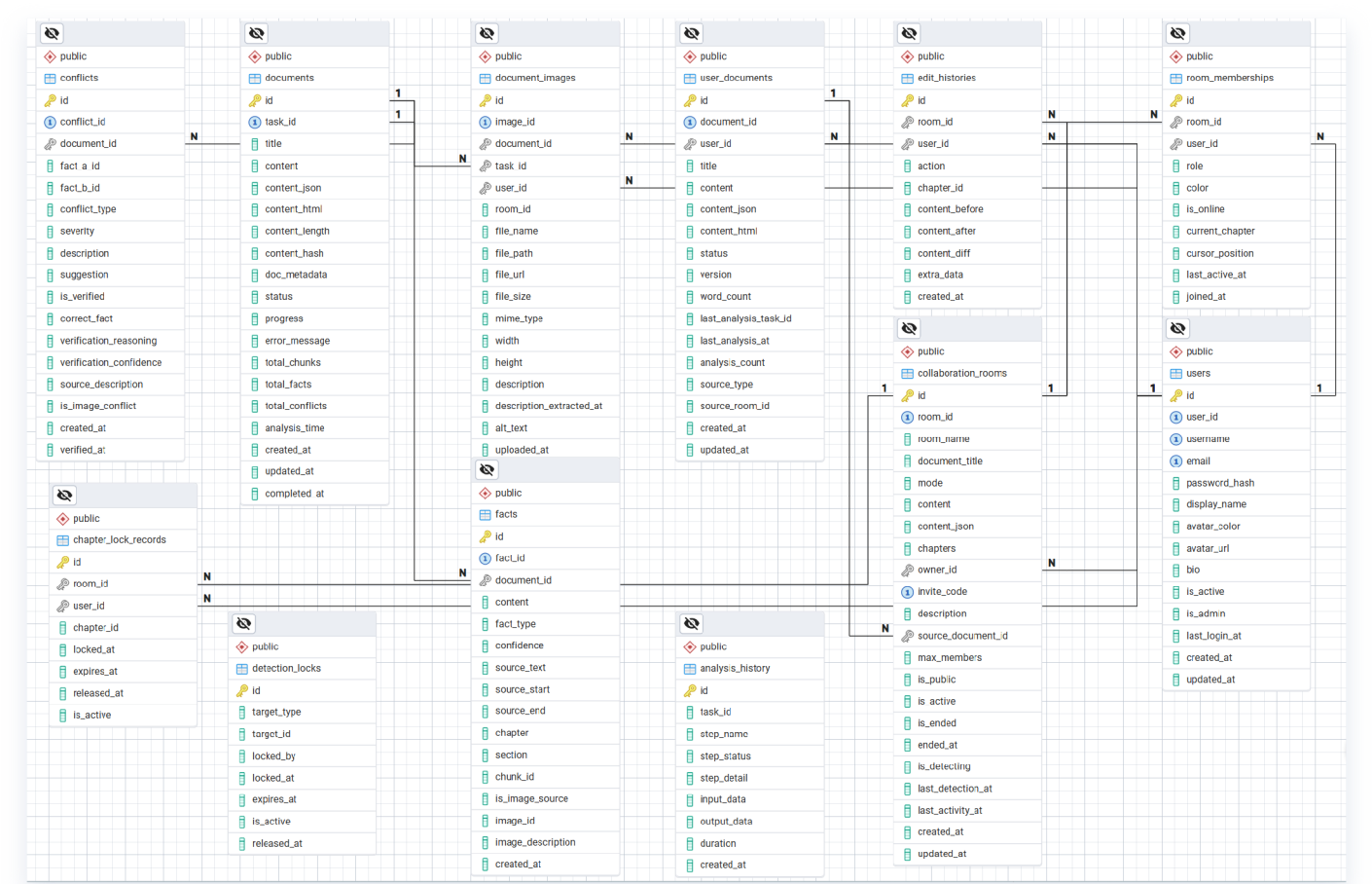
策略	实现方式	效果
输出约束	强制 JSON 格式输出 + Schema 验证	结构化输出，减少自由发挥
证据锚定	要求提供 <code>source_text</code> 原文依据	事实可追溯

策略	实现方式	效果
置信度评估	每个事实/冲突带置信度分数	低置信度结果标记警示
二次验证	溯源验证环节复核冲突	减少误报
温度控制	LLM temperature=0.1	降低随机性
事实过滤	过滤不值得检验的事实	聚焦关键信息

总体而言，上述策略覆盖“输出约束—证据可追溯—结果可复核”的关键链路，使系统在生成式模型不稳定的前提下仍具备可交付的稳定性。

七、数据库设计

7.1 PostgreSQL 表结构



7.2 Redis 数据结构

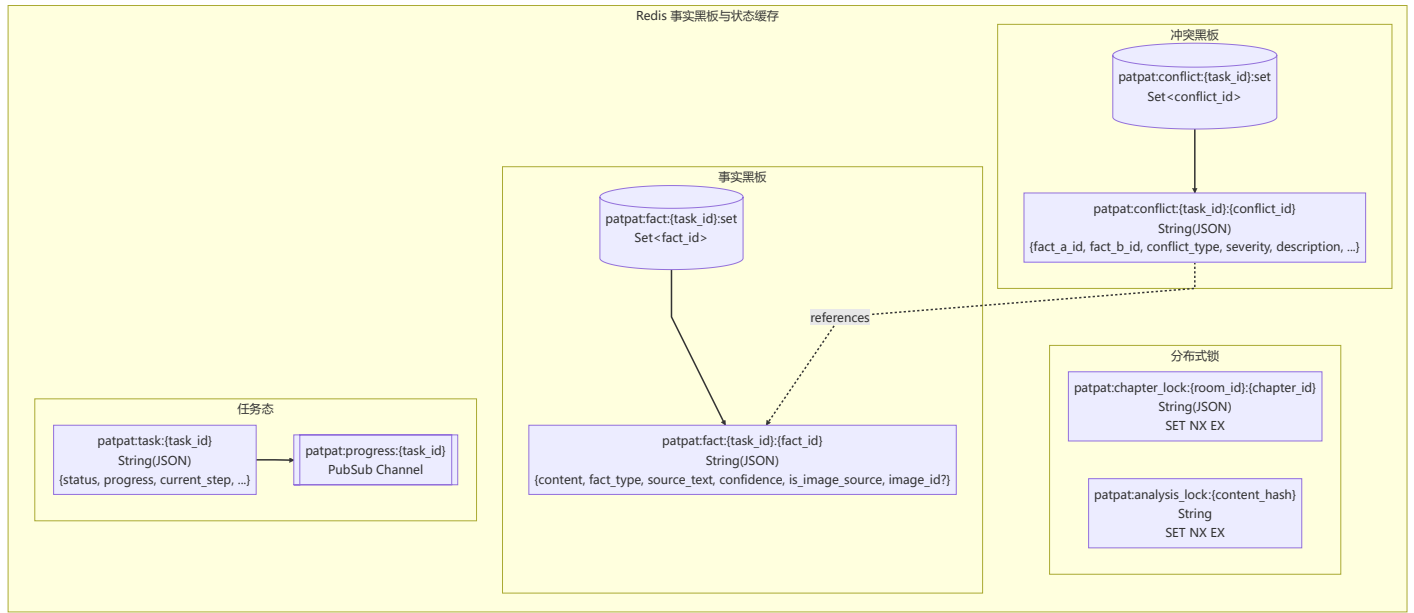
本系统将 Redis 定位为“事实黑板与状态缓存”的实时数据层：既承载分析任务的过程状态与进度事件，也承载事实/冲突的临时存储与协作锁，从而在长耗时任务与多人协作两类高并发场景下提供一致的读写语义。

为保证实现与验收口径一致，下表汇总了系统在 Redis 中使用的核心 Key 模式、数据类型与 TTL 设计，便于对“任务态、事实黑板、冲突黑板与协作锁”进行整体核对。

Key 模式	数据类型	描述	TTL
patpat:task:{task_id}	String(JSON)	任务状态（status/progress/current_step）	24h
patpat:progress:{task_id}	PubSub Channel	任务进度事件通道（WS 转发）	-
patpat:fact:{task_id}:{fact_id}	String(JSON)	单条事实详情	24h
patpat:fact:{task_id}:set	Set	任务事实 ID 集合	24h
patpat:conflict:{task_id}:{conflict_id}	String(JSON)	单条冲突详情	24h
patpat:conflict:{task_id}:set	Set	任务冲突 ID 集合	24h
patpat:chapter_lock:{room_id}:{chapter_id}	String(JSON)	章节锁（SET NX + EX，默认 5min）	5min
patpat:analysis_lock:{content_hash}	String	分析互斥锁（避免重复分析）	5min

通过把“状态、事实、冲突、锁”统一在同一键空间中，系统能够在长耗时任务与多人协作两类高并发场景下保持一致的数据读写与可观测能力。

为直观说明事实黑板的字段组织方式以及与冲突记录的引用关系，下面给出一个简化示例，用于展示单条事实、图片事实以及冲突记录在 Redis 中的存储形态。



基于上述键空间与数据形态，系统能够将“在线任务进度、事实/冲突可回溯、协作锁可观测”统一到一套可复用的工程实现中，为前端联动展示与实验取数提供稳定支撑。

八、Docker 容器化部署

8.1 服务编排

```
# docker-compose.yml
services:
  # 前端服务 (Nginx + React 静态资源)
  client:
    build: ./client
    ports: ["3000:80"]
    depends_on: [server]

  # 后端服务 (FastAPI + Uvicorn)
  server:
    build: ./server
    ports: ["8000:8000"]
    environment:
      - DEEPSEEK_API_KEY=${DEEPSEEK_API_KEY}
      - DATABASE_URL=postgresql://patpat:patpat123@postgres:5432/patpat_db
      - REDIS_HOST=redis
    volumes:
      - ./models:/app/models      # 视觉模型
      - ./uploads:/app/uploads    # 上传文件
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              capabilities: [gpu] # GPU加速
    depends_on: [redis, postgres]

  # Redis 事实黑板
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
    volumes: [redis-data:/data]
    command: redis-server --appendonly yes

  # PostgreSQL 持久化
  postgres:
```



```
image: postgres:15-alpine
ports: ["5433:5432"]
environment:
  - POSTGRES_USER=patpat
  - POSTGRES_PASSWORD=patpat123
  - POSTGRES_DB=patpat_db
volumes:
  - postgres-data:/var/lib/postgresql/data
  - ./server/init.sql:/docker-entrypoint-initdb.d/init.sql

networks:
  patpat-network:
    driver: bridge

volumes:
  redis-data:
  postgres-data:
```

8.2 部署流程

```
# 1. 克隆项目
git clone https://github.com/your-repo/PatPat-Inconsistency-Hunter.git
cd PatPat-Inconsistency-Hunter

# 2. 配置环境变量
cp env.example.txt .env
# 编辑 .env 填入 DEEPSEEK_API_KEY

# 3. 下载视觉模型到 models/ 目录
# InternVL3_5-4B-HF

# 4. 一键启动所有服务
docker compose up -d

# 5. 查看服务状态
docker compose ps

# 6. 查看日志
docker compose logs -f server

# 7. 访问应用
# 前端: http://localhost:3000
# API文档: http://localhost:8000/docs
```

我们已将系统部署到阿里云 ECS，核心思路是：保持 `docker-compose.yml` 的服务拆分不变，在云服务器上完成环境准备、端口开放与（可选）域名反向代理，即可把本地环境平滑迁移到公网。

阿里云部署建议按以下步骤执行：

- (1) **ECS 与网络准备**：创建 ECS（Linux 环境）并绑定公网 IP；在安全组放行必要端口（至少 `22/80/443`）。若短期不做反代，可临时放行 `3000/8000` 以完成联调与验收。
- (2) **运行环境安装**：在 ECS 上安装 Docker，并安装 Docker Compose 插件以支持一键编排启动。
- (3) **代码与依赖准备**：将仓库代码同步至 ECS；同时准备视觉模型目录（`models/`）与上传目录（`uploads/`），确保与 `docker-compose.yml` 中的挂载路径一致。
- (4) **环境变量配置**：创建 `.env` 并填入 `DEEPSEEK_API_KEY` 等配置项；密钥不写入仓库，避免泄露风险。
- (5) **启动与基础验收**：执行 `docker compose up -d` 启动所有服务，使用 `docker compose ps / docker compose logs` 检查容器状态；访问前端页面，并通过 `/docs` 验证后端接口可用。
- (6) **统一入口与反向代理（可选）**：为公网访问与 HTTPS 证书管理，通常使用 Nginx 监听 `80/443`，将 `/` 反代到前端容器（`3000`），将 `/api` 与 WebSocket 路径反代到后端容器（`8000`），以实现“单域名统一入口”。

通过以上流程，系统在本地与云端环境均可保持一致的部署形态。

九、前端界面展示

9.1 首页



首页承担“系统定位说明 + 功能入口汇聚”的作用，用于在用户进入系统时快速建立对能力边界的预期，并提供进入核心流程的统一入口。页面信息组织主要包括：

- **主操作区**：提供“开始分析 / 协作空间 / 仪表盘”等入口，将用户直接引导到高频业务流程。
- **能力卡片区**：以可量化的能力维度（支持字数、冲突类型、实时进度、部署形态等）进行概览展示，便于用户理解系统“能做什么/不能做什么”。
- **统一导航栏**：在顶部提供模块切换（首页/文档分析/协作空间/仪表盘/个人中心），保证跨页面跳转的一致体验。

9.2 文档分析页



文档分析页是“内容输入与任务发起”的主入口，围绕 `task_id` 组织前后端交互：用户提交文档后生成分析任务，前端再通过任务状态与进度推送完成过程展示与结果跳转。其核心交互可概括为：

- (1) **任务列表与历史回溯**：左侧展示历史分析任务，支持在多个 `task_id` 之间快速切换，保证实验取数与复核的可回溯性。

(2) **标题与富文本编辑**：支持输入标题、粘贴/编辑正文，并支持插入图片与预览，为后续图文一致性检测提供输入基础。

(3) **文件上传与解析回填**：支持 TXT/MD/DOCX/PDF 等格式解析，后端解析后回填到编辑器，便于用户在统一编辑器中二次修订。

(4) **任务创建与进度订阅**：点击开始分析后，前端调用 `POST /api/analyze` 创建任务并获取 `task_id`，随后通过 WebSocket `/api/ws/task/{task_id}/progress` 订阅进度事件，实时展示步骤文案与百分比。

9.3 分析结果页



分析结果页面向“人工复核与结果解释”的使用场景，目标是在不要求用户理解内部推理链路的前提下，让每一条事实与冲突都能回到原文定位并完成可验证的复核闭环。页面设计要点如下：

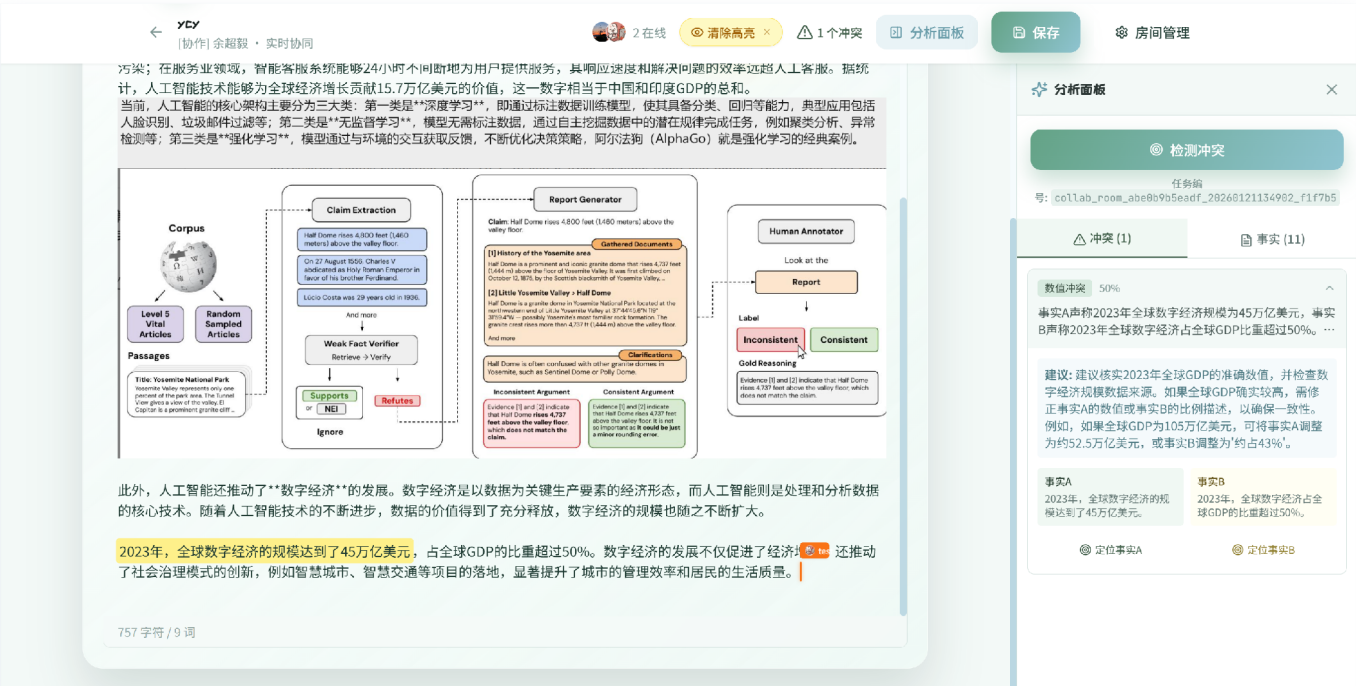
(1) **左右分栏结构**：左侧显示原文（含关键句高亮），右侧为结构化结果面板（事实/冲突 Tab），便于在“证据—结论”之间快速往返。

(2) **事实列表的可追溯展示**：事实条目包含类型、置信度与来源片段；用户点击事实后，左侧原文定位到对应 `source_text` 区域，降低核查成本。

(3) **冲突对照与建议**：冲突条目展示冲突类型、严重度、两条事实对照与修正建议；对图文冲突提供显式标识，提示用户重点关注“图表数据 vs 文本描述”的一致性。

(4) **任务摘要与导出留档**：页面顶部汇总字数、事实数、冲突数、耗时等指标，便于实验记录；同时支持通过 `GET /api/task/{task_id}/export` 导出结果，用于课程提交与留档。

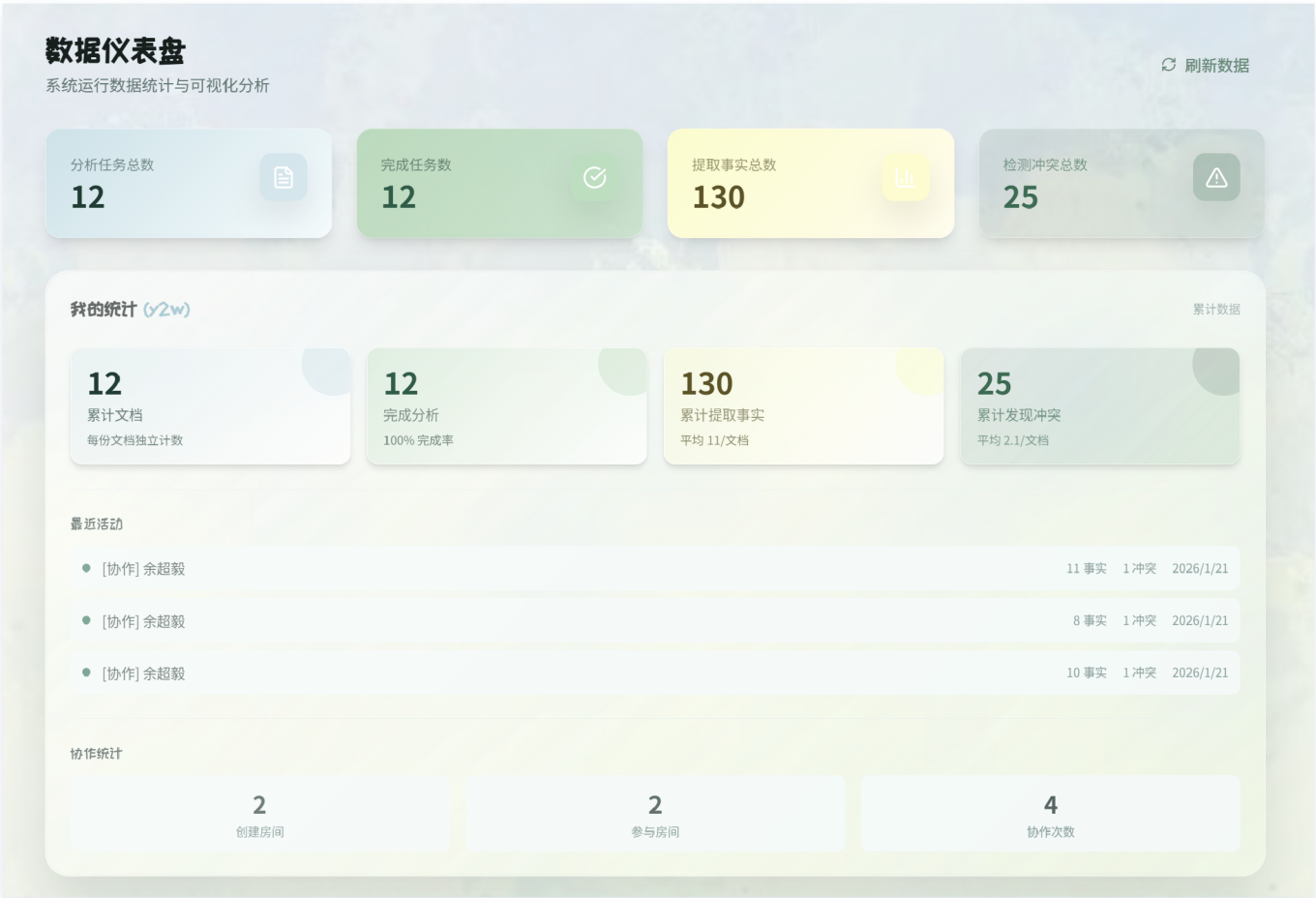
9.4 协作空间



协作空间面向“多人并发编辑同一文档”的真实需求，核心目标是在保证实时性体验的同时，避免内容覆盖与编辑冲突，并把冲突检测能力嵌入协作过程形成闭环。其主要能力包括：

- (1) **房间与成员管理**：支持创建/加入协作房间（如 `POST /api/collaboration/rooms`、`POST /api/collaboration/rooms/{room_id}/join`），并在前端展示成员列表与在线状态，便于协同分工。
- (2) **实时协同通信**：通过 WebSocket `/api/collaboration/ws/{room_id}/{user_id}/{username}` 广播内容变更与协作者光标/选区信息，降低多人编辑过程中的沟通成本。
- (3) **章节锁定机制**：在“分章节锁定模式”下，前端结合后端锁接口（如 `POST /api/collaboration/rooms/{room_id}/locks`）展示锁状态与占用信息，避免多人同时修改同一章节造成覆盖。
- (4) **协作内冲突检测**：在协作侧边栏触发检测（如 `POST /api/collaboration/rooms/{room_id}/detect`）并查看房间冲突结果（如 `GET /api/collaboration/rooms/{room_id}/conflicts`），使一致性问题能够在编辑过程中及时暴露与收敛。

9.5 统计仪表盘





统计仪表盘用于把离散的分析任务汇总为可解释的统计口径，辅助评估系统运行状态与检测效果，同时为课程展示提供直观的可视化支撑。页面功能要点如下：

- (1) **总体统计卡片**：汇总分析任务数、完成数、提取事实总量、检测冲突总量等关键指标，支撑“系统规模”层面的快速概览。
- (2) **分布与趋势分析**：展示冲突类型占比、严重度分布与近期趋势，便于观察任务量与冲突量变化，并为后续调参/迭代提供方向。
- (3) **个人/系统维度切换**：按用户维度查看个人历史分析表现，并可对照系统总体数据，常用数据接口包括 `GET /api/user/dashboard-stats` 与 `GET /api/stats/overview`。

9.6 个人中心/分析历史



个人中心页聚焦“用户维度的留痕与回溯”，用于将个人身份、统计指标与任务历史汇聚到同一入口，支撑课程展示与个人复盘：

- (1) **个人信息区**：展示头像/昵称等信息，并提供资料编辑与退出登录入口；身份信息通常通过 `GET /api/auth/me` 获取。
- (2) **个人统计摘要**：汇总累计分析次数、完成次数、累计事实、累计冲突等指标，便于快速展示个人使用情况（如 `GET /api/user/dashboard-stats`）。
- (3) **分析历史列表**：按时间列出历史任务，展示字数、事实数、冲突数与耗时等摘要，并支持跳转到分析结果页进行复核与导出留档。

十、实验结果与评估

我们使用同一篇测试文档（阿里云 GenAI 技术落地白皮书，约 6,420 字），对比五个不同系统在**文档内部冲突检测**任务上的表现。

10.1 指标定义

10.1.1 冲突查杀准确率 (Precision/Recall/F1)

系统输出的每条冲突视为一次“预测为冲突”。

- (1) **TP**: 系统报冲突, 人工判定确实冲突。
- (2) **FP**: 系统报冲突, 人工判定不冲突 (误报)。
- (3) **FN**: 人工判定有冲突, 但系统未报 (漏报)。

计算方式:

- (1) $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
- (2) $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$
- (3) $\text{F1} = 2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall})$

可复现评测流程 (推荐分层抽样, 避免某一类冲突占比过大导致偏差):

- 1. 从系统输出冲突按 8 种类型分层抽样 (每类 k 条, 不足则全量)。
- 2. 对每条样本人工标注“是否冲突/冲突类型是否正确/严重度是否合理”。
- 3. 另外在原文中人工标注若干“已知冲突点” (作为 FN 来源), 检查系统是否命中。

10.1.2 重复内容识别广度

重复内容识别的评估重点并非“是否命中某一句重复”, 而是系统是否能在事实层面完成去重聚合, 并在候选对生成阶段覆盖跨章节的同主题对照。对应实现主要体现在:

- (1) **事实去重聚合**: 事实抽取后进行去重, 减少重复事实带来的冗余 (`FactExtractor._deduplicate_facts(...)`)。
- (2) **跨章节候选对扩展**: 在候选对生成阶段引入跨章节匹配与实体重叠索引, 扩大“同一主题/实体”的对照范围 (`ConflictDetector._get_candidate_pairs(...)`)。

推荐采用以下量化口径, 以便与不同样例的规模差异解耦:

- (1) **去重率**: $\text{DedupRate} = 1 - (\text{去重后事实数} / \text{去重前事实数})$ 。
- (2) **重复点覆盖率**: 人工构造/标注 M 组“跨章节重复事实”, 统计系统成功抽取并纳入候选对的组数: $\text{Coverage_dup} = \text{Hit_dup} / M$ 。

通过同时观察去重率与覆盖率, 可区分“仅压缩冗余”与“覆盖跨章节对照”的能力边界, 为后续候选对策略迭代提供依据。

10.1.3 逻辑矛盾识别广度

逻辑矛盾识别对应冲突类型 `logical`（Prompt 有明确定义）。该类矛盾往往依赖跨句、跨段的语义前提，因此建议以人工构造/标注的逻辑矛盾点集为基准进行覆盖率评估：

- (1) 构造 `N_logic` 个逻辑矛盾点（如全称/特称矛盾、前提条件反转、结论互斥等），并在原文中标注证据位置。
- (2) 统计系统命中数 `Hit_logic`，并按覆盖率计算：`Coverage_logic = Hit_logic / N_logic`。

该指标用于衡量系统对“非数值类、非表面冲突”的识别覆盖能力，可与 10.1.1 的 Precision/Recall 形成互补：前者衡量命中质量，后者衡量覆盖广度。

10.2 实验设置与取数方式

为保证实验结果可复现，本项目采用“纯文本长文档 + 图文混排文档”两类样例进行评测，并统一使用后端接口进行指标取数与留档。

- (1) **样例 A（纯文本长文档）**：字数 ≥ 5000，章节 ≥ 6，人工植入数值/时间/实体/逻辑矛盾各 ≥ 1，用于验证系统在纯文本条件下的事实抽取稳定性与冲突识别能力。
- (2) **样例 B（图文混排）**：字数 ≥ 5000，图片 ≥ 3，至少 1 处“图文冲突”，用于验证视觉模型注入后的图文一致性检测能力。

取数口径建议统一通过以下接口完成，便于与结果记录表（10.3）对齐：

- 冲突列表：`GET /api/task/{task_id}/conflicts`
- 事实列表：`GET /api/task/{task_id}/facts`
- 任务摘要：`GET /api/task/{task_id}/result`
- 导出报告：`GET /api/task/{task_id}/export?format=markdown`

10.3 结果记录表

10.3.1 检测结果总览对比

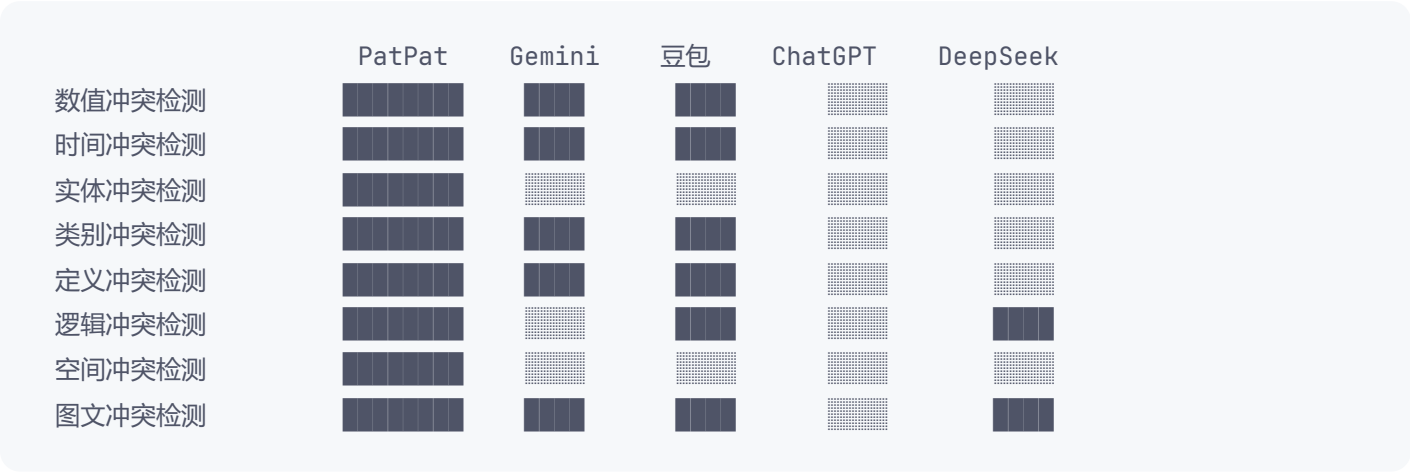
指标	🏆 PatPat	Gemini	豆包	ChatGPT	DeepSeek
检测到的冲突总数	38	6	6	0	2
提取的事实数量	73	-	-	-	-
覆盖冲突类型数	8/8	5/8	6/8	0/8	2/8
提供验证推理	✅	❌	❌	❌	❌
提供修正建议	✅	❌	❌	❌	❌

指标	🏆 PatPat	Gemini	豆包	ChatGPT	DeepSeek
置信度评估	✅	❌	❌	❌	❌
图文冲突检测	✅	✅	✅	❌	⚠️
原文溯源定位	✅	✅	✅	❌	✅

10.3.2 各类型冲突检测数量对比

冲突类型	🏆 PatPat	Gemini	豆包	ChatGPT	DeepSeek
数值冲突	2	1	1	0	0
时间冲突	1	1	1	0	0
实体冲突	3	0	0	0	0
类别冲突	4	1	1	0	0
定义冲突	12	2	1	0	0
逻辑冲突	7	0	1	0	1
空间冲突	1	0	0	0	0
图文冲突	8	1	1	0	1
合计	38	6	6	0	2

10.3.3 检测能力雷达图对比



10.3.4 关键冲突检测对比（实例分析）

1. 数值冲突：大模型成本比例 (50% vs 1%)

系统	检测结果
🏆 PatPat	✅ 准确检测：识别出"直接使用大模型成本是深度研发的50%"与"仅为1%"的严重数值矛盾，严重程度评估90%，并提供验证推理和修正建议
Gemini	✅ 检测到，但无验证分析
豆包	✅ 检测到，但无验证分析

系统	检测结果
ChatGPT	✗ 未检测到
DeepSeek	✗ 未检测到

2. 时间冲突：ChatGPT 面世日期

系统	检测结果
🏆 PatPat	✅ 准确检测：识别"2022年11月30日"与"2023年3月"的时间矛盾，并验证正确答案为2022年11月30日，置信度90%
Gemini	✅ 检测到
豆包	✅ 检测到
ChatGPT	✗ 未检测到
DeepSeek	✗ 未检测到

3. 图文冲突：AI Agent 工具能力

系统	检测结果
🏆 PatPat	✅ 深度检测：发现文本声称"不具备使用工具的能力"与图6中明确包含"工具"模块（日历、计算器等）的严重矛盾，并进行多角度验证
Gemini	✅ 检测到
豆包	✅ 检测到
ChatGPT	✗ 未检测到（声称"文档不包含图片"）
DeepSeek	⚠ 标记为"可能"，不确定

4. 定义冲突：RAG 概念混淆

系统	检测结果
🏆 PatPat	✅ 多层次检测：发现 RAG 被错误定义为"AI Agent/智能实体"与正确定义"检索增强生成技术"的冲突，检测到7处相关定义不一致
Gemini	✅ 检测到2处
豆包	✅ 检测到1处
ChatGPT	✗ 未检测到
DeepSeek	✗ 未检测到

5. 逻辑冲突：基础设施部署方式

系统	检测结果
 PatPat	✅ 深度检测：发现"自建数据中心成本高"与"自建数据中心更经济"的逻辑矛盾，共检测到7处逻辑冲突
Gemini	❌ 未检测到
豆包	✅ 检测到1处
ChatGPT	❌ 未检测到
DeepSeek	✅ 检测到1处

10.3.5 性能量化分析

- 检测召回率对比（以 PatPat 检测结果为基准）

系统	检测数量	召回率	评价
 PatPat	38	100%	基准系统
Gemini	6	15.8%	仅检测到部分显著冲突
豆包	6	15.8%	仅检测到部分显著冲突
ChatGPT	0	0%	完全漏检
DeepSeek	2	5.3%	大量漏检

- 冲突类型覆盖度

系统	覆盖类型	覆盖率
 PatPat	8/8	100%
Gemini	5/8	62.5%
豆包	6/8	75%
ChatGPT	0/8	0%
DeepSeek	2/8	25%

十一、API 接口设计

本系统 API 采用“REST + WebSocket”组合：

- (1) REST 负责任务创建、结果查询、导出与协作管理，接口风格保持幂等与可重试，便于前端与自动化取数脚本复用。
- (2) WebSocket 负责长耗时分析任务的进度推送与协作实时事件广播，避免前端轮询造成无效负载。

下表仅列出与课程验收/评测取数最相关的接口（更细粒度接口在前端 `client/src/api/index.js` 中有完整封装）。

11.1 文档分析接口

文档分析接口以 `task_id` 为核心索引：创建任务后，前端通过状态/结果/事实/冲突等接口逐步拉取数据；同时通过进度 WebSocket 实时更新进度条与步骤提示。

为方便前后端联调与评测取数，下表列出文档分析链路中最关键的接口与用途说明。

方法	路径	描述
POST	<code>/api/analyze</code>	提交文档分析任务
GET	<code>/api/task/{task_id}/status</code>	获取任务状态
GET	<code>/api/task/{task_id}/result</code>	获取分析结果
GET	<code>/api/task/{task_id}/facts</code>	获取事实列表
GET	<code>/api/task/{task_id}/conflicts</code>	获取冲突列表
WS	<code>/api/ws/task/{task_id}/progress</code>	任务进度推送（实时更新进度条）
GET	<code>/api/task/{task_id}/export</code>	导出分析报告
POST	<code>/api/task/{task_id}/reanalyze</code>	重新分析
POST	<code>/api/parse-document</code>	解析上传文件

上述接口覆盖从“任务创建—过程可观测—结果可回溯—数据可导出”的完整闭环，能够满足前端交互展示与实验取数的双重需求。

11.2 协作接口

协作接口同时覆盖“房间管理、实时协同、章节锁、协作内冲突检测”。其中 WebSocket 用于广播内容变更、光标位置、锁状态等事件，保证多人编辑时的实时一致性体验。

下表汇总协作场景下的核心接口，涵盖房间生命周期、实时通信与锁管理等关键能力。

方法	路径	描述
POST	<code>/api/collaboration/rooms</code>	创建协作房间
POST	<code>/api/collaboration/rooms/{room_id}/join</code>	加入房间
WS	<code>/api/collaboration/ws/{room_id}/{user_id}/{username}</code>	WebSocket连接
POST	<code>/api/collaboration/rooms/{room_id}/detect</code>	触发冲突检测

方法	路径	描述
GET	/api/collaboration/rooms/{room_id}/conflicts	获取房间冲突
POST	/api/collaboration/rooms/{room_id}/locks	获取章节锁

通过接口层对协作过程进行显式建模（房间、成员、锁、事件），系统能够在多人并发编辑下保持内容一致性与冲突结果的可追踪。

11.3 用户接口

用户接口用于支撑登录鉴权与统计看板：既包含个人维度的 dashboard 数据，也包含系统维度的总览数据，用于课程展示与实验取数。

为便于展示系统使用情况与分析产出，下表列出用户侧最常用的鉴权与统计接口。

方法	路径	描述
POST	/api/auth/register	用户注册
POST	/api/auth/login	用户登录
GET	/api/auth/me	获取当前用户
GET	/api/user/dashboard-stats	用户统计
GET	/api/stats/overview	系统统计概览

这些接口与前述任务/协作接口配合，构成“用户身份—任务产出—统计可视化”的数据闭环。

十二、项目分工

本项目以“后端/算法 Agent/前端”三条主线并行推进，并在接口对齐与联调阶段进行交叉协作：算法侧负责输出结构与质量，工程侧负责稳定落地与可观测性，前端侧负责交互闭环与可视化呈现。

为明确项目组织方式与工作边界，下表对成员角色、负责模块与贡献占比进行汇总。

成员	学号	角色	负责模块	贡献占比
余超毅	10235501470	算法/Agent组	LangGraph工作流、Prompt工程、事实提取、冲突检测、溯源验证、视觉模型集成	33%

成员	学号	角色	负责模块	贡献占比
吴彤	10222140442	架构/工程组	系统架构设计、 Docker部署、数据库设计、Redis集成、API开发	33%
王惜冉	10235501401	前端/交互组	React前端开发、富文本编辑器、协作功能、可视化仪表盘、UI/UX设计	33%

通过“主线分工 + 联调交叉”的组织方式，项目在保证模块内聚的同时，也能够接口与验收阶段快速完成协同与收敛。

十三、总结

13.1 项目成果

本项目成功实现了一个基于 LLM Agent 的长文本事实一致性检测与协作系统，完成了从“文档解析 → 事实抽取 → 冲突检测 → 溯源验证 → 可视化与协作”的端到端闭环。

功能覆盖与产出物：

为便于从验收口径快速核对系统实现范围，下表汇总了本项目的核心能力与对应落地情况。

指标	实现情况
✔ 事实提取	支持 6 种事实类型（数值/时间/实体/定义/结论/关系），并支持图片来源事实（ <code>image</code> 前缀）
✔ 冲突检测	支持 8 种冲突类型 + 四级严重度评估
✔ 溯源校验	二次验证机制，输出修正建议与置信度
✔ 图片识别	InternVL3_5-4B-HF 本地部署，支持图文一致性检测
✔ 协作编辑	实时协同 + 分章节锁定两种协作模式
✔ 云原生部署	Docker Compose 一键部署，服务拆分清晰
✔ 事实黑板	Redis 统一承载实时状态、事实/冲突黑板与分布式锁
✔ 可视化仪表盘	统计图表 + 分析历史 + 冲突分布展示

可以看到，系统已经覆盖从“事实抽取—冲突推理—结果呈现—协作保障—部署落地”的关键环节，满足课程实验与演示所需的完整闭环。

13.2 技术亮点

本节从“可控性、覆盖率、可用性”三个维度归纳系统亮点：既要保证 LLM 推理的可控与可回溯，也要在成本可控的前提下尽可能扩大识别范围，并最终落实到可交付的前端体验与可复现评测。

1. LangGraph workflow编排（可控、可回滚、可扩展）

将长文档一致性检测拆为图结构节点，工程侧可以明确定位失败节点并做降级输出；同时节点之间通过状态传递，天然适配“部分成功、部分失败仍可交付”的场景。

- （1）支持条件分支：例如跳过验证（ `skip_verification` ）时仍输出可用冲突结果。
- （2）支持节点失败隔离：图片识别失败不影响纯文本流程；验证失败不阻断冲突检测结果落库。

2. 候选对生成策略（在成本约束下提升覆盖率）

冲突检测若做全量两两比对复杂度高、成本不可控。本系统通过候选对生成将“可能冲突”的事实优先送入 LLM，对低价值组合直接剪枝，把算力与 token 用在高价值对照上。

- 配合事实去重，减少重复事实导致的候选膨胀，提升检测效率与“重复内容识别广度”。

3. 多模态理解（图文一致性检测）

通过视觉模型将图片信息转为文本描述并注入统一流程，使系统不仅能看“文字内部自洽”，也能做“文字 vs 图表/表格数据”的一致性检查，覆盖更贴近真实报告场景的矛盾类型。

4. Prompt 工程与幻觉防护（结构化、可验证、可追溯）

在报告输出层面，最关键的是让模型“说得清、查得到、错了能兜底”。因此系统用结构化输出约束与 JSON 修复兜底来提升鲁棒性，并保留溯源信息让用户可复核。

5. 事实黑板与分布式锁（并发一致性与协作实时性）

Redis 既承载任务状态与事实/冲突黑板，也承载协作章节锁，从而把“长任务状态一致性”和“多人实时协作一致性”统一到一套可观测的数据结构上。

6. 可观测性与用户体验

系统通过进度推送让长耗时任务“可见”，通过原文高亮联动让结果“可用”，最终把模型能力转化为可交付的交互闭环。